

Conceptos básicos para la programación de aplicaciones Android.

Caso práctico

Una vez que el equipo de trabajo tiene el IDE instalado y algunas ideas de cómo se pueden programar las apps y probarlas en dispositivos virtuales y reales, es necesario adquirir los conceptos básicos en el desarrollo de las aplicaciones para Android.



[Mikhail](#) ([Pexels](#))

- Buenos días -saluda Alda.
- Buenos días -responden María, Juan y Pedro.
- Hoy comenzaremos viendo las distintas fases de la programación por las que es necesario pasar para convertir nuestro código java en una aplicación ejecutable en un terminal Android -indica Ada-. Después estudiaremos las peculiaridades de la **interfaz de usuario** de las aplicaciones Android, los **controles principales** y el **manejo de sus eventos** más importantes, así como algunos conceptos interesantes para su diseño como el manejo del **foco**.
- Perfecto -responde Juan-. Además seguro que los conceptos previos que tenemos sobre Programación Orientada a Objetos nos van a ser muy útiles.
- Por supuesto -afirma Ada-. Es muy importante conocer los distintos **controles de entrada** de datos que nos van a permitir interactuar con nuestros usuarios. Os voy a asignar algunos pequeños proyectos para que entendáis su funcionamiento.
- ¡Genial! -exclama María.



[Ministerio de Educación y Formación Profesional](#), (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

1.- Fases de la programación.

Caso práctico



[Mikhail](#) ([Pexels](#))

Todos los usuarios de dispositivos móviles sabemos que cuando deseamos instalar una app en nuestro dispositivo acudimos a la "tienda" de nuestro proveedor. En caso de dispositivos Apple abriremos *Apple Store* y en caso de dispositivos Android abriremos *Google Play Store*.

Estas plataformas son en realidad repositorios de paquetes que albergan las aplicaciones.

- Como ya imagináis, desde que se desarrolla una app hasta que se publica en uno de estos repositorios de publicación, es necesario pasar por una serie de fases -explica Ada-. Estas fases forman el ciclo de vida de una app móvil.
- Entiendo que como nosotros vamos a desarrollar aplicaciones para Android, el repositorio de publicación que le corresponde es Google Play Store. - pregunta Pedro-. ¿Es así?.
- Estás en lo cierto, Pedro -confirma Ada-. En el caso de Android, el fichero de manifiesto (**Androidmanifest.xml**) juega un papel muy importante en la definición de los metadatos asociados a la app y al propio paquete.
- ¿Cuáles son esos metadatos? -pregunta María.
- Son, por ejemplo, las actividades que forman la aplicación, los permisos que se necesitan y otra gran cantidad de datos relativos a la propia aplicación - responde Ada.
- ¡Pues vamos a ver las fases de la programación! -exclama sonriente Juan.

En la unidad anterior ya se ha avanzado algo sobre las herramientas que podrían utilizarse para desarrollar una aplicación para dispositivo móvil. Como ya has debido ver anteriormente en el módulo de **Programación**, normalmente una aplicación Java podrá ser desarrollada utilizando herramientas a través de la línea de comandos o bien mediante la utilización de un entorno de desarrollo integrado (**IDE**). Sea cual sea el método que se utilice para el desarrollo del código, podrás distinguir las siguientes fases de construcción para una aplicación basada en Java con **Android Studio**:

- ✓ Desarrollo del código
- ✓ Compilación
- ✓ Preverificación
- ✓ Empaquetamiento
- ✓ Ejecución

✔ Depuración

En nuestro caso, el entorno de desarrollo que usaremos será Android Studio. Por otro lado, en la unidad anterior ya vimos cómo instalar esta herramienta para poder desarrollar aplicaciones Android y cómo compilar una aplicación básica para poder ejecutarla en el emulador y en un dispositivo real.

1.1.- Desarrollo del código: codificación.

Esta es la fase en la que vas a escribir las líneas de **código** de tu aplicación. Si no estuvieras utilizando un IDE (en este caso Android Studio) necesitarías un editor de texto para escribir y almacenar los archivos java que contendrán las clases, así como los ficheros XML que se van a ir generando. En este caso, la cosa es tan sencilla como utilizar el editor de textos proporcionado por Android Studio.

Si el entorno integrado de desarrollo dispone de asistentes para la creación de aplicaciones, parte de ese código podrá ser generado automáticamente por el asistente, especialmente todo lo que concierne a la interfaz gráfica de usuario y especificación de recursos que utilizará el sistema. Android Studio dispone de estos asistentes y los usaremos con frecuencia para facilitarnos el trabajo.

1.2.- Compilación y preverificación.

El proceso de **compilación**, es decir, de generación de un archivo binario de tipo bytecode (class) a partir de otro en código **Java** ya lo has estudiado en otros módulos y no cambia en absoluto. Una vez que tengas el código fuente de tu programa escrito en uno o varios archivos con extensión **.java**, la compilación de aplicaciones se puede realizar con el programa *javac* incluido en el software de desarrollo de Java o bien mediante la utilización del entorno integrado de Android Studio.

Los procesos de **verificación** durante la generación de código se encargan de realizar revisiones de seguridad, integridad y cumplimiento de los estándares. Debido a las importantes restricciones existentes en los dispositivos móviles, la **verificación** de las clases tiene dos etapas:

- ✔ **Preverificación**, que se realiza como un paso más de la generación de los ficheros binarios.
- ✔ **Verificación**, llevada a cabo por la máquina virtual durante la ejecución.

1.3.- Empaquetamiento.

Esta fase es la que se encarga de empaquetar la aplicación con todas las clases y recursos que vaya a necesitar, dejándola lista para ser descargada en un dispositivo que la instale.

Si tratamos de generar una aplicación de Android tendremos que exportar nuestro proyecto Android Studio a un fichero [APK](#)  (*Android Application Package*, significado en español: *Paquete de Aplicación Android*). El nombre APK se refiere a la extensión que tiene el fichero compacto y empaquetado que contiene todo el código de la aplicación.

En este proceso de exportación hay dos alternativas posibles según como se haya realizado el proceso:

- a. Generación de Fichero **APK** de tipo **Debug**. Consistirá en un fichero APK destinado a poder hacer de manera local pruebas de funcionamiento de la aplicación, instalándolo en dispositivos virtuales o en nuestro propio dispositivo físico, pero todo ello en un ámbito restringido. En ningún caso este fichero es válido para subirlo a **Play Store Google**, aún así si podré instalarlo de manera remota en mi dispositivo móvil siempre que haya habilitado el dispositivo para permitir instalación de aplicaciones **no firmadas**.
- b. Fichero APK firmado digitalmente (**signed APK**). Consistirá en un fichero que ha sido firmado con un certificado digital, de manera que este fichero garantiza la autoría del mismo por la persona que desarrolló la aplicación. Este tipo de fichero si puede ser subido a **Play Store Google** para publicar una app. De hecho, si no está firmado no puede subirse y cualquier actualización que hagamos de la app en Play Store, deberá ser firmada con el mismo certificado siempre.

1.4.- El fichero de manifiesto.

Cuando generamos cualquier aplicación, siempre llevará incorporada un fichero de "manifiesto" que contiene información de configuración de la aplicación en cuestión. Este archivo describe el contenido de la aplicación, indicando las actividades que contiene, así como entre otras cosas permisos para acceder a ciertos recursos y funciones del dispositivo, y consiste un archivo de texto con una estructura de una línea por atributo, donde los valores de los atributos son especificados de la forma "atributo: valor".

Según [Android Developer](#)  :

Todas las aplicaciones deben tener un fichero **AndroidManifest.xml** (con ese nombre exacto) en el directorio *nombre_proyecto/app/src/main*. El archivo de manifiesto proporciona información esencial sobre tu aplicación al sistema Android, información que el sistema debe tener para poder ejecutar el código de la app.

Entre otras cosas, el fichero de manifiesto hace lo siguiente:

- ✓ Nombra el paquete de Java para la aplicación. El nombre del paquete sirve como un identificador único para la aplicación. Cuando lo subas a Play Store Google debe ser único globalmente.
- ✓ Describe los componentes de la aplicación, como las actividades, los servicios, los receptores de mensajes y los proveedores de contenido que la integran. También nombra las clases que implementa cada uno de los componentes y publica sus capacidades, como los mensajes [Intent](#)  con los que pueden funcionar. Estas declaraciones notifican al sistema Android los componentes y las condiciones para el lanzamiento.
- ✓ Declara los permisos que debe tener la aplicación para acceder a las partes protegidas de una API e interactuar con otras aplicaciones. También declara los permisos que otros deben tener para interactuar con los componentes de la aplicación.
- ✓ Funciones Hardware y Software que requiere la aplicación, lo que implica que se pueda instalar o no en un dispositivo.
- ✓ Enumera las clases [Instrumentation](#)  que proporcionan un perfil y otra información mientras la aplicación se ejecuta. Estas declaraciones están en el manifiesto solo mientras la aplicación se desarrolla y se quitan antes de la publicación de esta.

Debes conocer

Además del archivo `AndroidManifest.xml` existen otros archivos importantes como el archivo `nombre_proyecto/app/build.gradle` donde se indican algunos parámetros importantes de la aplicación Android

- ✓ Declara una lista de `_____plugins` que deseamos incluir en la aplicación.
- ✓ Declara varios parámetros importantes de la aplicación, algunos de ellos:
 - ➡ Identificador (es el nombre del paquete java) y versión de la aplicación.
 - ➡ Versión del SDK con la que se va a realizar la compilación.

- ➡ Versión mínima de SDK necesaria en el terminal para que la app funcione y la versión objetivo (*target*) para la que se ha diseñado.
- ✓ Enumera las bibliotecas de las que la aplicación depende (*dependencies*).

```
android {
    namespace "com.example.myapplication"
    compileSdk 32
    defaultConfig {
        applicationId "com.example.myapplication"
        minSdk 26
        targetSdk 32
        versionCode 1
        versionName "1.0"
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
}

dependencies {
    implementation 'androidx.appcompat:appcompat:1.4.2'
    implementation 'com.google.android.material:material:1.6.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    testImplementation 'junit:junit:4.13.2'
    androidTestImplementation 'androidx.test.ext:junit:1.1.3'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.4.0'
}
```

[Google Developers](#) (Uso educativo nc)

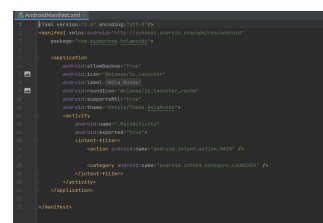
1.5.- Estructura del fichero de manifiesto.

A continuación se muestra la estructura general de dicho fichero (todos los enlaces indicados en el código llevan a enlaces de Android Developer y se abren en ventana nueva).

Estructura general del fichero de manifiesto

```
<?xml version="1.0" encoding="utf-8"?>
<manifest>
  <uses-permission /> <permission /> <permission-tree /> <permission-group /> <instru
  <supports-screens /> <compatible-screens /> <supports-gl-texture />
  <application>
    <activity>
      <intent-filter>
        <action />
        <category />
        <data />
      </intent-filter>
      <meta-data />
    </activity>
    <activity-alias>
      <intent-filter> . . . </intent-filter>
      <meta-data />
    </activity-alias>
    <service>
      <intent-filter> . . . </intent-filter>
      <meta-data/>
    </service>
    <receiver>
      <intent-filter> . . . </intent-filter>
      <meta-data />
    </receiver>
    <provider>
      <grant-uri-permission />
      <meta-data />
      <path-permission />
    </provider>
    <uses-library />
  </application>
</manifest>
```

En la estructura anterior se presentan todas las posibles etiquetas que podrían utilizarse. En la práctica usaremos muchas menos. Así, en una aplicación básica, con una solo actividad (ficheros asociados **MainActivity.java** y **activity_main.xml**) el fichero **AndroidManifest.xml** presenta este aspecto, como vemos, bastante más simple:



La siguiente lista contiene todos los elementos que pueden aparecer en el fichero de manifiesto (en orden alfabético):

- ✓ [<action>](#) 
- ✓ [<activity>](#) 
- ✓ [<activity-alias>](#) 
- ✓ [<application>](#) 
- ✓ [<category>](#) 
- ✓ [<data>](#) 
- ✓ [<grant-uri-permission>](#) 
- ✓ [<instrumentation>](#) 
- ✓ [<intent-filter>](#) 
- ✓ [<manifest>](#) 
- ✓ [<meta-data>](#) 
- ✓ [<permission>](#) 
- ✓ [<permission-group>](#) 
- ✓ [<permission-tree>](#) 
- ✓ [<provider>](#) 
- ✓ [<receiver>](#) 
- ✓ [<service>](#) 
- ✓ [<supports-screens>](#) 
- ✓ [<uses-configuration>](#) 
- ✓ [<uses-feature>](#) 
- ✓ [<uses-library>](#) 
- ✓ [<uses-permission>](#) 
- ✓ [<uses-sdk>](#) 

Nota: Estos son todos los elementos permitidos; no puedes agregar elementos o atributos propios.

1.6.- Convenciones del fichero de manifiesto.

A todos los elementos y atributos del fichero de manifiesto se les aplican una serie de convenciones y reglas generales:

Elementos

Solo se requieren los elementos `<manifest>` y `<application>`. Cada uno de ellos debe estar presente y solo puede ocurrir una vez. La mayoría de los otros elementos pueden ocurrir varias veces o ninguna. Sin embargo, al menos algunos de ellos deben estar presentes para que el archivo de manifiesto sea útil.

Si un elemento contiene algo, serán otros elementos. Todos los valores se establecen mediante atributos, en lugar de datos de caracteres en un elemento.

Por lo general, los elementos de un mismo nivel no están ordenados. Por ejemplo, los elementos `<activity>`, `<provider>` y `<service>` se pueden alternar en cualquier secuencia. Hay dos excepciones importantes a esta regla:

- ✓ Un elemento `<activity-alias>` debe seguir el `<activity>` en el que funciona como alias.
- ✓ El elemento `<application>` debe ser el último dentro del elemento `<manifest>`. En otras palabras, la etiqueta de cierre de `</application>` debe aparecer inmediatamente antes de la etiqueta de cierre de `</manifest>`.

Atributos

En términos formales, todos los atributos son opcionales. Sin embargo, existen algunos atributos que deben especificarse para que un elemento cumpla con su propósito. Usa la documentación como guía. Para los atributos realmente opcionales, se menciona un valor predeterminado o se indica lo que sucede si no se especifica ningún valor.

Excepto para algunos atributos del elemento raíz `<manifest>`, todos los nombres de los atributos comienzan con un prefijo **android:**. Por ejemplo, **android:alwaysRetainTaskState**. Debido a que el prefijo es universal, en la documentación suele omitirse cuando se hace referencia a los atributos por nombre.

Declaración de nombres de clases

Muchos elementos corresponden a objetos Java, incluidos los elementos para la aplicación en sí (el elemento `<application>`) y sus componentes principales: actividades (`<activity>`), servicios (`<service>`), receptores de mensajes (`<receiver>`) y proveedores de contenido (`<provider>`).

Si defines una subclase, como lo harías generalmente para las clases de componentes ([Activity](#) , [Service](#) , [BroadcastReceiver](#)  y [ContentProvider](#) ), la subclase se declara mediante un atributo **name**. El nombre debe incluir la designación completa del paquete.

Ejemplo

Una subclase [Service](#)  podría declararse de la siguiente manera:

```
1 | <manifest . . . >
2 |     <application . . . >
3 |         <service
4 |             android:name="com.example.project.SecretService" . . . >
5 |             . . .
6 |         </service>
7 |         . . .
8 |     </application>
9 | </manifest>
```

Sin embargo, si el primer carácter de la cadena (*string*) es un punto, el nombre del paquete de la aplicación (tal como lo especifica el atributo [package](#)  del elemento [<manifest>](#) ) se adjunta a la cadena (*string*).

Ejemplo

La siguiente asignación es la misma que se mostró en el ejemplo anterior.

```
1 | <manifest package="com.example.project" . . . >
2 |     <application . . . >
3 |         <service
4 |             android:name=".SecretService" . . . >
5 |             . . .
6 |         </service>
7 |         . . .
8 |     </application>
9 | </manifest>
```

Cuando inicia un componente, el sistema Android crea una instancia de la subclase mencionada. Si no se especifica una subclase, crea una instancia de la clase base.

Varios valores

Si se puede especificar más de un valor, el elemento se repite casi siempre, en lugar de enumerar varios valores en un único elemento.

Ejemplo

En un **filtro de objetos Intent** se pueden enumerar varias acciones:

```
1 <intent-filter. . . >
2     <action android:name="android.intent.action.EDIT" />
3     <action android:name="android.intent.action.INSERT" />
4     <action android:name="android.intent.action.DELETE" />
5 </intent-filter>
```

Valores de recursos

Algunos atributos tienen valores que se pueden mostrar a los usuarios, como una etiqueta y un ícono para una actividad. Los valores de estos atributos deben localizarse y establecerse desde un recurso o tema. Los valores de los recursos se expresan en el siguiente formato:

@[<i>package</i>:]<i>type</i>/<i>name</i>

Puedes omitir el nombre de paquete (*package*) si el recurso se encuentra en el mismo paquete de la aplicación.

type es un tipo de recurso, como una cadena (*string*) o un elemento de diseño (*drawable*).

name es el nombre que identifica al recurso específico.

A continuación, te mostramos un ejemplo:

<activity android:icon="@drawable/smallPic" . . . >

Los valores de un tema se expresan de manera similar, aunque con un ? al inicio en lugar de @:

?[<i>package</i>:]<i>type</i>/<i>name</i>

Valores de cadenas (strings)

Cuando el valor de un atributo sea una cadena (*string*), debes usar doble barra invertida (\\) para diferenciarlo de los caracteres, como en el caso del uso de \\n para una nueva línea o \\uxxxx para un carácter Unicode.

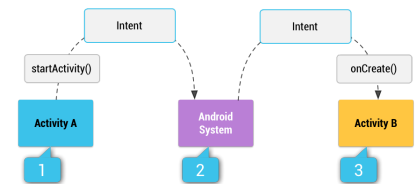
1.7.- Funciones del fichero de manifiesto.

Filtros de intenciones (*intent filters*)

Los componentes centrales de una aplicación (como sus actividades, servicios y receptores de mensajes) se activan mediante **intenciones** (*intents*). Las intenciones se representan mediante la clase [Intent](#) .

Un objeto de mensajería **intent**:

- ✓ Es una instancia de la clase **Intent** y podemos usarlo para solicitar una acción desde otro componente de la app.
- ✓ Facilita la comunicación entre componentes de distintas maneras.
- ✓ Describe una acción deseada, los datos en los que se debe actuar, la categoría del componente que debe realizar la acción y otras instrucciones pertinentes.
- ✓ Es respondido apropiadamente porque Android localiza un componente para ello, iniciando una instancia nueva del mismo si es necesario y le pasa el objeto de mensajería (ver [Intent](#) ).



[Google Developers](#) (Uso educativo nc)

Los componentes anuncian los tipos de objetos de mensajería (**intents**) a los que pueden responder mediante **filtros de intenciones** (*intents*). Debido a que el sistema Android debe aprender las intenciones (*intents*) que puede manejar un componente antes de iniciarlo, los **filtros de intenciones** (*intents*) se especifican en el manifiesto como elementos [<intent-filter>](#) . Un componente puede tener cualquier cantidad.

Ejemplo

Ejemplo de filtro de intenciones (declaración de actividad con un filtro de intenciones para recibir una intención ACTION_SEND cuando el tipo de datos es texto):

```
1 <activity android:name="ShareActivity">
2   <intent-filter>
3     <action android:name="android.intent.action.SEND"/>
4     <category android:name="android.intent.category.DEFAULT"/>
5     <data android:mimeType="text/plain"/>
6   </intent-filter>
7 </activity>
```

Una **intención** (*intent*) que nombre explícitamente un componente de destino activará ese componente; el filtro no participa. Una **intención** (*intent*) que no nombra explícitamente un

componente de destino puede activar un componente únicamente si pasa por uno de los filtros del componente.

El uso más importante de las **intenciones** (*intents*) es para el lanzamiento de actividades, donde se puede pensar que son el "pegamento" entre las **actividades** (*activities*).

Por tanto, una **intención** (*intent*) es una estructura de datos pasiva que representa una descripción abstracta de una acción a ser realizada.

Íconos y etiquetas

Varios elementos tienen atributos **icon** y **label** para un icono pequeño y una etiqueta de texto que se pueden mostrar a los usuarios. Algunos también tienen un atributo **description** para un texto explicativo más extenso que también se puede mostrar en pantalla. Por ejemplo, el elemento `<permission>`  tiene estos tres atributos. Cuando se pregunta al usuario si otorga permiso a una aplicación que lo solicitó, se le proporciona un icono que representa el permiso, el nombre del permiso y una descripción de lo que implica.

En cada caso, el icono y la etiqueta establecidos en un elemento contenedor se convierten en la configuración predeterminada de **icon** y **label** para todos los subelementos del contenedor. Por lo tanto, el icono y la etiqueta establecidos en el elemento `<application>`  son el icono y la etiqueta predeterminados para cada componente de la aplicación. Igualmente, el icono y la etiqueta establecidos para un componente, como un elemento `<activity>` , son la configuración predeterminada de cada elemento `<intent-filter>`  del componente. Si un elemento `<application>`  establece una etiqueta, pero una actividad y su filtro de intenciones (*intents*) no lo hacen, la etiqueta de la aplicación se considera como la etiqueta de la actividad y del filtro de intenciones (*intents*).

El icono y la etiqueta establecidos para un filtro de intenciones (*intents*) se usan para representar un componente cuando el componente se presenta al usuario y cumple la función indicada en el filtro. Por ejemplo, un filtro con las configuraciones **android.intent.action.MAIN** y **android.intent.category.LAUNCHER** indica que la actividad inicia una aplicación. Es decir, como una que se debe mostrar en el lanzador de la aplicación. El icono y la etiqueta establecidos en el filtro se muestran en el lanzador.

Permisos

Un *permiso* es una restricción que limita el acceso a una parte del código o a datos en el dispositivo. La limitación se impone para proteger datos y códigos claves que podrían usarse incorrectamente para distorsionar o afectar la experiencia del usuario.

Cada permiso se identifica con una etiqueta única. Con frecuencia, la etiqueta indica la acción que está restringida. A continuación, se nombran algunos permisos definidos por Android:

- ✓ `android.permission.CALL_EMERGENCY_NUMBERS`
- ✓ `android.permission.READ_OWNER_DATA`
- ✓ `android.permission.SET_WALLPAPER`
- ✓ `android.permission.DEVICE_POWER`

Una funcionalidad se puede proteger con un solo permiso.

Si una aplicación necesita acceso a una característica protegida por un permiso, debe declarar que requiere el permiso con un elemento [<uses-permission>](#)  en el manifiesto. Cuando la aplicación se instala en el dispositivo, el instalador determina si otorgará el permiso solicitado controlando las autoridades que firmaron los certificados de la aplicación y, en algunos casos, preguntándole al usuario. Si se otorga el permiso, la aplicación puede usar las características protegidas. De lo contrario, los intentos de acceder a esas características fallarán y no habrá notificaciones para el usuario.

Una aplicación también puede proteger sus propios componentes con permisos. Puede emplear cualquiera de los permisos definidos por Android (enumerados en [android.Manifest.permission](#) ) o declarados por otras aplicaciones. También puede definir sus propios permisos. Un permiso nuevo se declara con el elemento [<permission>](#) .

Ejemplo

Una actividad se puede proteger de la siguiente manera:

```
1  <manifest . . . >
2      <permission android:name="com.example.project.DEBIT_ACCT" . . . />
3      <uses-permission android:name="com.example.project.DEBIT_ACCT" />
4      . . .
5      <application . . . >
6          <activity android:name="com.example.project.FreneticActivity"
7                  android:permission="com.example.project.DEBIT_ACCT"
8                  . . . >
9          . . .
10     </activity>
11 </application>
12 </manifest>
```

Ten en cuenta que, en este ejemplo, el permiso **DEBIT_ACCT** no solo se declara con el elemento `<permission>`; su uso también se solicita con el elemento `<uses-permission>`. Debes solicitar su uso para que otros componentes de la aplicación inicien la actividad protegida, aunque la protección se imponga a través de la propia aplicación.

En el mismo ejemplo, si el atributo **permission** estuviera establecido en un permiso declarado en otro lugar (como **android.permission.CALL_EMERGENCY_NUMBERS**), no sería necesario volver a declararlo con un elemento `<permission>`. Sin embargo, seguiría siendo necesario solicitar su uso con `<uses-permission>`.

El elemento [<permission-tree>](#)  declara un espacio de nombres para un grupo de permisos definidos en código y [<permission-group>](#)  define una etiqueta para un conjunto de permisos (aquellos declarados en el manifiesto con elementos `<permission>` y aquellos declarados en otro lugar). Esto solo afecta la agrupación de los permisos cuando se presentan al usuario. El elemento `<permission-group>` no especifica los permisos pertenecientes al grupo, pero le asigna el nombre. Puedes disponer un permiso en el grupo si asignas el nombre de este al elemento `<permission>` del atributo `permission-group`.

Bibliotecas

Todas las aplicaciones están vinculadas con la biblioteca predeterminada de Android, que incluye los paquetes básicos para crear aplicaciones (con clases comunes como **Activity**, **Service**, **Intent**, **View**, **Button**, **Application** y **ContentProvider**).

Sin embargo, algunos paquetes se encuentran en sus propias bibliotecas. Si tu aplicación usa código de alguno de estos paquetes, debe solicitar explícitamente su vinculación con ellos. El manifiesto debe contener un elemento `<uses-library>`  separado para nombrar cada una de las bibliotecas. En la documentación del paquete puedes encontrar el nombre de la biblioteca.

1.8.- Instalación en un dispositivo real.

Para la instalación de una aplicación en un dispositivo real existen varias alternativas, pero has de tener en cuenta que en el fondo se trata de copiar el archivo APK de la aplicación en la memoria del dispositivo y realizar su instalación en él.

Si el archivo APK se encuentra en un servidor, sería suficiente con descargarlo mediante el navegador web del dispositivo o bien utilizar un gestor de aplicaciones y a partir de ese momento sería el propio dispositivo el que se encargaría de descargar e instalar el archivo APK.

Si el archivo APK no está en un servidor, sino que lo tienes en tu ordenador, recién compilado y en espera de pruebas, puedes copiarlo a la memoria del dispositivo de la manera que te resulte más cómoda:

- ✔ Mediante la transferencia del archivo APK por Bluetooth que integra el propio dispositivo.
- ✔ A través de una conexión USB y copiando el archivo APK en la memoria del dispositivo.
- ✔ Copiando el archivo APK en alguna memoria externa soportada por el dispositivo (una tarjeta SD por ejemplo) e insertándole esa memoria.

1.9.- Ejecución.

La ejecución de una aplicación puede realizarse bien en un emulador (basta con ejecutarla en Android Studio para que ésta sea lanzada en el emulador que tengamos configurado en el proyecto) bien en un dispositivo real. Es casi seguro que el aspecto final será distinto en ambos casos, aunque el funcionamiento debería ser prácticamente idéntico (a veces podría haber algunas diferencias debido a la máquina virtual que haya instalada en el dispositivo). También es muy probable que haya cambios de aspecto entre distintos dispositivos (dependiendo de la máquina virtual que tengan instalada).

En el momento en que la aplicación se ponga a funcionar se irán sucediendo cada uno de los estados **en pausa**, **activo** y **destruido** y se irán ejecutando todos los métodos que hayamos añadido.

2.- Interfaces de usuario.

Caso práctico

Una vez que el equipo es conocedor de las fases que tendrá que completar para convertir el programa desarrollado con Android Studio en una aplicación descargable en un dispositivo, es el momento de pensar en el diseño gráfico que permitirá a los usuarios usar la aplicación.



[Mikhail](#) ([Pexels](#))

- El diseño gráfico de una aplicación es muy importante -explica Ada-. El diseño será lo que conseguirá, o no, que nuestra aplicación sea usable en distintos dispositivos y permita la interacción con el usuario.
- Claro -responde Juan-, un mal diseño gráfico puede hacer inservible una aplicación.
- Exacto -confirma Ada-. Para ello necesitáis conocer algunos parámetros del diseño o layouts.
- ¿Qué lenguaje se utiliza para el diseño de las aplicaciones Android? - pregunta Juan.
- Vas a tener suerte -responde Ada-, todo el diseño (layout) de la aplicación Android se configura mediante el lenguaje XML.
- ¡Qué bien! -exclama Juan-, pondré en práctica todo lo aprendido en el módulo de Lenguaje de Marcas.

2.1.- Información general de la Interfaz de usuario.

La interfaz gráfica de usuario es la parte de un programa que permite al usuario interactuar con él. En el caso de Java SE dispones de dos extensas bibliotecas gráficas (AWT y Swing, que son parte de las JFC), pero para el caso de Android Studio o Java ME, dada la gran cantidad de restricciones que tienen los dispositivos móviles, hubo que diseñar un pequeño conjunto de clases específicas que cumplieran esas restricciones.

La interfaz de usuario (IU) de tu app es todo aquello que el usuario puede ver y todo aquello con lo que este puede interactuar. Android ofrece una variedad de componentes de IU previamente compilados, como objetos de diseño estructurados y controles de IU que te permiten compilar la interfaz gráfica de usuario para tu app. Android también ofrece otros módulos de IU para interfaces especiales, como diálogos, notificaciones y menús.

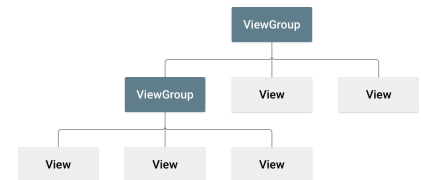
Según Android Developers todos los elementos de la interfaz de usuario de una app para Android están desarrollados con objetos [View](#)  y [ViewGroup](#) . Un objeto **View** dibuja algo en la pantalla con lo que el usuario puede interactuar. Un objeto **ViewGroup** agrupa otros objetos **View** (y **ViewGroup**) para definir el diseño de la interfaz.

Android proporciona una colección de subclases **View** (y **ViewGroup**) que te ofrecen controles de entrada comunes (como los botones y los campos de texto) y varios modelos de diseño (como un diseño lineal o relativo).

2.2.- Diseño de la interfaz de usuario.

El diseño (*layout*) define la estructura de la interfaz de usuario de tu app. Todos los elementos en el *layout* se construyen usando una jerarquía de objetos [View](#)  y [ViewGroup](#) , como se muestra en la ilustración más abajo.

Una vista (objeto View) normalmente dibuja algo que el usuario puede ver y con lo que puede interactuar mientras que un grupo de vista (objeto ViewGroup) es un contenedor invisible que define la estructura de diseño de otros objetos View y ViewGroup. Este árbol de jerarquía puede ser tan simple o complejo como lo necesites (pero la simplicidad es lo mejor para el rendimiento).



[Google Developers](#) (Uso educativo nc)

Los objetos View se denominan controles (*widgets*) y podemos crear instancias de muchas subclases como por ejemplo [Button](#)  o [TextView](#) . Los objetos ViewGroup se denominan estructuras de diseño (*layouts*) y pueden ser de distintos tipos los cuales proporcionan diferentes estructuras de diseño como [LinearLayout](#)  o [ConstraintLayout](#) .

Para definir el diseño, puedes crear instancias de objetos ViewGroup y View en el código y desarrollar un árbol, pero la manera más sencilla y efectiva de definir el diseño consiste en utilizar un archivo XML. XML ofrece una estructura en lenguaje natural para el diseño, similar a HTML.

El nombre de un elemento XML para una vista se relaciona con la clase de Android que representa. Por lo tanto, un elemento `<TextView>` crea un control gráfico (**widget**) TextView en la IU, y un elemento `<LinearLayout>` crea un grupo de vista LinearLayout.

Ejemplo

Un diseño vertical simple con una vista de texto y un botón tiene esta apariencia:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="fill_parent"
4      android:layout_height="fill_parent"
5      android:orientation="vertical" >
6      <TextView android:id="@+id/text"
7          android:layout_width="wrap_content"
8          android:layout_height="wrap_content"
9          android:text="Soy una vista de texto" />
10     <Button android:id="@+id/button"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="Soy un botón" />
14 </LinearLayout>
```

Cuando cargas un recurso de diseño en la app, Android inicia cada nodo del diseño en un objeto de tiempo de ejecución que puedes usar para definir comportamientos adicionales, consultar el estado del objeto o modificar el diseño.

Para consultar una guía completa sobre la creación de un diseño de IU, consulta el apartado siguiente sobre Diseños (layouts).

2.3.- Otros componentes de la interfaz de usuario.

Además de los objetos **View** y **ViewGroup**. Android proporciona varios componentes de app que ofrecen un diseño de IU estándar, en los cuales solo tienes que definir el contenido. Cada uno de estos componentes de IU tienen un conjunto único de clases en la API y se describen en los apartados de la documentación correspondientes, como [Adición de la barra de app](#)  , [Cuadros de diálogo](#)  y [Notificaciones de estado](#)  .

2.4.- Diseños (layouts).

Un diseño (*layout*) define la estructura visual para una interfaz de usuario, como la IU para una **actividad** o control (*widget*) de una app. Puedes definir un diseño (*layout*) de dos maneras:

- ✓ **Definir elementos de la IU en XML.** Android proporciona un vocabulario XML simple que coincide con las clases y subclases de vistas, como las que se usan para controles (*widgets*) y diseños.
- ✓ **Instanciar elementos de diseño en tiempo de ejecución.** Tu aplicación puede crear objetos **View** y **ViewGroup** (y manipular sus propiedades) programáticamente.

El framework de Android te ofrece la flexibilidad de usar uno de estos métodos o ambos para definir y administrar la IU de tu aplicación. Por ejemplo, podrías definir los diseños predeterminados de la aplicación en XML, incluidos los elementos de pantalla que aparecerán en ellos y sus propiedades. Luego podrías agregar código en tu aplicación para modificar el estado de los objetos de la pantalla, incluidos los declarados en XML, en tiempo de ejecución.

La ventaja de definir tu IU en XML es que te permite separar mejor la presentación de tu aplicación del código que controla su comportamiento. Tus descripciones de la IU son externas al código de tu aplicación, lo que significa que puedes modificarlo o adaptarlo sin tener que modificar el código fuente y volver a compilar. Por ejemplo, puedes crear diseños XML para diferente orientación de pantalla, diferentes tamaños de pantalla de dispositivos y diferentes idiomas. Además, definir el diseño en XML facilita la visualización de la estructura de tu IU, de modo que sea más fácil depurar problemas.

En general, el vocabulario XML para definir elementos de la IU sigue de cerca la estructura y la denominación de las clases y los métodos, en los que los nombres de los elementos coinciden con los nombres de las clases y los nombres de los atributos coinciden con los métodos. De hecho, la correspondencia generalmente es tan directa que puedes deducir qué atributo XML corresponde a un método de clase, o deducir qué clase corresponde a un elemento XML determinado. No obstante, ten en cuenta que no todo el vocabulario es idéntico. En algunos casos, hay pequeñas diferencias de denominación. Por ejemplo, el elemento [EditText](#)  tiene un atributo **text** que puede asignarse usando el método [setText\(\)](#) .

En los siguientes apartados trataremos aspectos importantes del diseño.

2.5.- Escribe en XML.

Al usar vocabulario XML de Android, puedes crear rápidamente diseños de IU y de los elementos de pantalla que contienen, de la misma manera que creas páginas web en HTML, con una serie de elementos anidados.

Cada fichero de diseño debe contener exactamente un elemento raíz, que debe ser un objeto [View](#) o [ViewGroup](#). Una vez que hayas definido el elemento raíz, puedes agregar controles (**widgets**) u objetos de diseño adicionales como elementos secundarios para crear gradualmente una jerarquía de vistas que defina tu diseño.

Ejemplo

Aquí te mostramos un diseño XML que usa un elemento [LinearLayout](#) (subtipo de ViewGroup) vertical para incluir un elemento [TextView](#) (subtipo de View) y un elemento [Button](#) (subtipo de TextView):

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:orientation="vertical" >
6     <TextView android:id="@+id/text"
7         android:layout_width="wrap_content"
8         android:layout_height="wrap_content"
9         android:text="Hola, soy una vista de texto" />
10    <Button android:id="@+id/button"
11        android:layout_width="wrap_content"
12        android:layout_height="wrap_content"
13        android:text="Hola, soy un botón" />
14 </LinearLayout>
```

Después de definir tu diseño en XML, guarda el fichero con la extensión **.xml** en el directorio **res/layout/** de tu proyecto de Android para que pueda compilarse correctamente.

2.6.- Carga el recurso XML.

Cuando compilas tu aplicación, cada fichero de diseño XML se compila en un recurso View. Debes cargar el recurso de diseño desde el código de tu aplicación, en tu implementación (redefinición) del método `onCreate()` de la clase Activity. Para hacerlo, llama a `setContentView()`, pásale la referencia a tu recurso de diseño en forma de: `R.layout.layout_file_name`.

Ejemplo

Si tu diseño XML se guarda como **main_layout.xml**, lo cargarías para tu actividad de esta manera:

```
1 public void onCreate(Bundle savedInstanceState) {  
2     super.onCreate(savedInstanceState);  
3     setContentView(R.layout.main_layout);  
4 }
```

El framework de Android llama al método de retrollamada **onCreate()** en tu actividad cuando se lanza la actividad.

2.7.- Atributos.

Todos los objetos **View** y **ViewGroup** admiten su propia variedad de atributos XML. Algunos atributos son específicos para un objeto **View** (por ejemplo, **TextView** admite el atributo **textSize**), pero a esos atributos también los heredan otros objetos **View** que podrían extender esta clase. Algunos son comunes para todos los objetos **View** ya que se heredan desde la clase **View** raíz (como el atributo **id**). Otros atributos se consideran "parámetros de diseño" y son atributos que describen ciertos tipos de orientación de diseño del objeto **View**, tal como lo define el objeto principal **ViewGroup** de ese objeto.

2.8.- ID.

Cualquier objeto **View** puede tener un **id** con número entero asociado a él para identificar de forma exclusiva la vista en un árbol. Cuando se compila la aplicación, este **id** se considera un número entero, pero el **id** generalmente se asigna en el archivo XML del diseño como un string, en el atributo **id**. Este es un atributo XML común a todos los objetos View (definidos por la clase [View](#) ) y lo usarás con mucha frecuencia. La sintaxis para un **id** dentro de una etiqueta XML es la siguiente:

```
android:id="@+id/my_button"
```

El símbolo arroba (@) al comienzo del string indica al analizador de XML que debe analizar y expandir el resto de la cadena de **id** e identificarla como un recurso de **id**. El símbolo más (+) significa que es un nuevo nombre de recurso que se debe crear y agregar a nuestros recursos (en el archivo **R.java**). El framework de Android ofrece otros recursos de **id**. Al hacer referencia a un **id** de recurso de Android, no necesitas el símbolo más, pero debes agregar el espacio de nombres de paquete **android** de la siguiente manera:

```
android:id="@android:id/empty"
```

Con el espacio de nombres de paquete **android** establecido, ahora hacemos referencia a un **id** de la clase de recursos **android.R**, en lugar de la clase de recursos local.

Para crear vistas y hacer referencia a ellas desde la aplicación, puedes seguir este patrón común:

1. Definir una vista o un control (**widget**) en el archivo de diseño y asignarle un **id** único:

Código

```
1 <Button android:id="@+id/my_button"
2         android:layout_width="wrap_content"
3         android:layout_height="wrap_content"
4         android:text="@string/my_button_text"/>
```

2. Luego, crear una instancia del objeto **View** y capturarlo desde el diseño (generalmente en el método `onCreate()`):

```
Button myButton = (Button) findViewById(R.id.my_button);
```

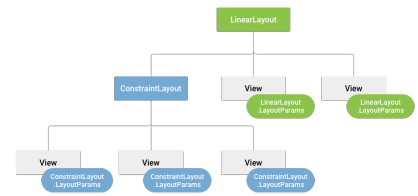
Definir **id** para objetos **View** es importante cuando se crea un [RelativeLayout](#) . En un diseño relativo, las vistas del mismo nivel pueden definir su diseño en función de otra vista del mismo nivel, que se identifica con un **id** único.

No es necesario que un **id** sea único en todo el árbol, pero debe ser único dentro de la parte del árbol en la que estás buscando (que a menudo puede ser el árbol completo, por lo que es mejor que, en lo posible, sea totalmente único).

2.9.- Parámetros de diseño.

Los atributos de diseño XML denominados **layout_something** definen parámetros de diseño para el objeto **View** que son adecuados para el objeto **ViewGroup** en el que reside.

Cada clase **ViewGroup** implementa una clase anidada que extiende [ViewGroup.LayoutParams](#) . Esta subclase contiene tipos de propiedad que definen el tamaño y la posición de cada vista secundaria, según resulte apropiado para el grupo de vistas. Como puedes ver en la figura, el grupo de vistas principal define parámetros de diseño para cada vista secundaria (incluido el grupo de vistas secundario).



[Google Developers](#) (Uso educativo nc)

Todos los grupos de vistas incluyen un ancho y una altura (**layout_width** y **layout_height**), y cada vista debe definirlos. Muchos **LayoutParams** también incluyen márgenes y bordes opcionales.

Puedes especificar el ancho y la altura con medidas exactas, aunque probablemente no quieras hacerlo con mucha frecuencia. Generalmente usarás una de estas constantes para establecer el ancho o la altura:

- ✓ **wrap_content** indica a tu vista que modifique su tamaño conforme a los requisitos de este contenido.
- ✓ **match_parent** indica a tu vista que se agrande tanto como lo permita su grupo de vistas principal.

En general, no se recomienda especificar el ancho y la altura de un diseño con unidades absolutas como píxeles. En cambio, el uso de medidas relativas como **unidades de píxeles independientes de la densidad (dp)**, **wrap_content**, o **match_parent**, es un mejor enfoque, ya que ayuda a garantizar que tu aplicación se muestre correctamente en dispositivos con pantallas de diferentes tamaños.

2.10.- Posición del diseño.

La geometría de una vista es la de un rectángulo. Una vista tiene una ubicación, expresada como un par de coordenadas *izquierda* y *superior*, y dos dimensiones, expresadas como un ancho y una altura. La unidad para la ubicación y las dimensiones es el pixel.

Es posible recuperar la ubicación de una vista al invocar los métodos `getLeft()`  y `getTop()` . El primero devuelve la coordenada izquierda, o X, del rectángulo que representa la vista. El segundo devuelve la coordenada superior, o Y, del rectángulo que representa la vista. Ambos métodos devuelven la ubicación de la vista respecto de su elemento primario. Por ejemplo, cuando `getLeft()` devuelve 20, significa que la vista se encuentra a 20 píxeles a la derecha del borde izquierdo de su elemento primario directo.

Además, se ofrecen varios métodos convenientes para evitar cálculos innecesarios, y se denominan `getRight()`  y `getBottom()` . Estos métodos devuelven las coordenadas de los bordes derecho y superior del rectángulo que representa la vista. Por ejemplo, llamar a `getRight()` es similar al siguiente cálculo: `getLeft() + getWidth()`.

2.11.- Tamaño, relleno y márgenes.

El tamaño de una vista se expresa con un ancho y una altura. En realidad, una vista tiene dos pares de valores de ancho y altura.

El primer par se conoce como *ancho medido* y *altura medida*. Estas dimensiones definen cuán grande quiere ser una vista dentro de su elemento primario. Las dimensiones medidas se pueden obtener llamando a [getMeasuredWidth\(\)](#)  y a [getMeasuredHeight\(\)](#) .

El segundo par se conoce simplemente como *ancho* y *altura*, o algunas veces *ancho de dibujo* y *altura de dibujo*. Estas dimensiones definen el tamaño real de la vista en la pantalla, al momento de dibujarlas y después del diseño. Estos valores pueden ser diferentes del ancho y la altura medidos, pero no necesariamente. El ancho y la altura se pueden obtener llamando a [getWidth\(\)](#)  y [getHeight\(\)](#) .

Para medir estas dimensiones, una vista considera su relleno. El relleno se expresa en píxeles para las partes izquierda, superior, derecha e inferior de la vista. El relleno se puede usar para desplazar el contenido de la vista una determinada cantidad de píxeles. Por ejemplo, un relleno izquierdo de 2 empuja el contenido de la vista 2 píxeles hacia la derecha del borde izquierdo. El relleno se puede ajustar usando el método [setPadding\(int, int, int, int\)](#)  y se puede consultar llamando a [getPaddingLeft\(\)](#) , [getPaddingTop\(\)](#) , [getPaddingRight\(\)](#)  y [getPaddingBottom\(\)](#) .

Si bien una vista puede definir un relleno, no proporciona ningún tipo de soporte para márgenes. No obstante, los grupos de vistas sí lo proporcionan. Consulta [ViewGroup](#)  y [ViewGroup.MarginLayoutParams](#)  para obtener más información.

2.12.- Diseños comunes.

Cada subclase de la clase [ViewGroup](#)  proporciona una manera única de mostrar las vistas que anidas en ella. Aquí te mostramos algunos de los tipos de diseño más comunes integrados en la plataforma Android y que podemos ver en la documentación de Android Developer.

Nota: Si bien puedes anidar uno o más diseños dentro de otro diseño para crear la presentación de tu IU, debes esforzarte por mantener tu jerarquía de diseño lo más sencilla posible. Tu diseño dibuja más rápido si tiene menos diseños anidados (una jerarquía de vistas ancha es mejor que una jerarquía de vista profunda).



[Google Developers](#) (Uso educativo nc)

1.- [Diseño lineal](#)

Un diseño que organiza sus elementos secundarios en una sola fila horizontal o vertical. Si la longitud de la ventana supera la longitud de la pantalla, crea una barra de desplazamiento.

2.- [Diseño relativo](#)

Te permite especificar la ubicación de los objetos secundarios en función de ellos mismos (el objeto secundario A a la izquierda del objeto secundario B) o en función del elemento primario (alineado con la parte superior del elemento primario).



[Google Developers](#) (Uso educativo nc)



[Google Developers](#) (Uso educativo nc)

3.- [Vista web](#)

Muestra páginas web.

3.- Controles de entrada y sus eventos.

Caso práctico

Además de conocer los distintos parámetros de diseño y cómo definirlos en una app de Android Studio mediante el lenguaje XML y los ficheros de recursos (carpeta res), es imprescindible conocer los mecanismos de la plataforma para interactuar con los usuarios.

- ¿Alguno sabe decirme cómo se llama lo que provoca un usuario que interactúa con un control? -pregunta Ada.

- Un evento -responde María-. Le he leído en la documentación que nos has entregado.

- Muy bien María -le felicita Ada-. Realmente, cuando un usuario interactúa con un control provoca que se lancen eventos. Es necesario, en el diseño de la aplicación, aprender a capturar los eventos y programar las funciones que van a responder a los mismos.

- Otra pregunta -vuelve a preguntar Ada-, ¿cómo se llaman las funciones que responden a los eventos?

- Controladores de eventos -responde Pedro.

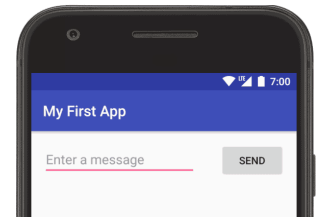
- Veo que habéis estudiado la documentación -sonríe Ada-. Vamos a aprender cómo capturar los eventos y programar sus controladores.



[Mikhail](#) ([pexels](#))

3.1.- Controles de entrada.

Consultando la página de Android Developer podemos descubrir que los controles de entrada son los componentes interactivos de la interfaz de usuario de tu app. Android ofrece una gran variedad de controles que puedes usar en tu IU, como botones, campos de texto, barras de búsqueda, casillas de verificación, botones de zoom, botones de activación o desactivación y muchos más.



[Google Developers](#) (Uso educativo nc)

Agregar un control de entrada a tu IU es tan simple como agregar un elemento XML a tu [diseño XML](#) .

Ejemplo

Aquí te mostramos un diseño con un campo de texto y un botón:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="fill_parent"
4      android:layout_height="fill_parent"
5      android:orientation="horizontal">
6
7      <EditText android:id="@+id/edit_message"
8          android:layout_weight="1"
9          android:layout_width="0dp"
10         android:layout_height="wrap_content"
11         android:hint="@string/edit_message" />
12
13     <Button android:id="@+id/button_send"
14         android:layout_width="wrap_content"
15         android:layout_height="wrap_content"
16         android:text="@string/button_send"
17         android:onClick="sendMessage" />
18
19 </LinearLayout>
```

Cada control de entrada admite un conjunto específico de eventos de entrada de modo que puedas gestionar eventos, como cuando el usuario introduce texto o pulsa un botón.

3.1.1.- Controles comunes.

Aquí te ofrecemos una lista con algunos de los controles comunes que puedes usar en tu app. Sigue los vínculos para obtener más información acerca de cómo usar cada uno.

Tipo de control	Clases relacionadas	Descripción
Botón 	Button 	Botón que el usuario puede presionar o en el que puede hacer clic para realizar una acción
Campo de texto 	EditText  AutoCompleteTextView 	Campo de texto editable. Puedes usar el control <code>AutoCompleteTextView</code> para permitir al usuario introducir texto con sugerencias de autocompletado
Casilla de verificación 	CheckBox 	Conmutador de selección y deselección que el usuario puede activar o desactivar. Debes usar casillas de verificación (<i>checkboxes</i>) cuando presentes a los usuarios un grupo de opciones seleccionables que no sean mutuamente exclusivas.
Botón de selección 	RadioGroup  RadioButton 	Similar a las casillas de verificación, excepto porque solo se puede seleccionar una opción en el grupo.
Botón para activar o desactivar 	ToggleButton 	Botón de activación y desactivación con un indicador luminoso.
Control de número 	Spinner 	Una lista desplegable que permite a los usuarios seleccionar un valor numérico de un conjunto.
Selectores 	DatePicker  TimePicker 	Un cuadro de diálogo para que los usuarios seleccionen un valor individual para un conjunto con los botones de arriba y abajo o mediante un gesto de deslizamiento. Usa un control <code>DatePicker</code> para leer valores de fecha (mes, día, año) o un control <code>TimePicker</code> para leer valores de hora (hora, minuto) en modo 24 horas o AM/PM, a los que se les dará formato automáticamente de acuerdo con la configuración regional del usuario.

3.2.- Eventos de entrada.

Según Android Developer:

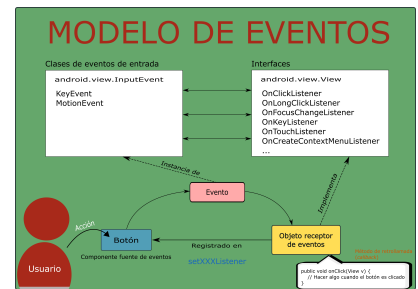
En Android, existe más de una forma de interceptar los eventos provocados por una interacción del usuario con tu aplicación. Al considerar los eventos dentro de tu interfaz de usuario, el enfoque consiste en capturar los eventos desde el objeto de vista específico con el que el usuario interactúa. La clase **View** proporciona los medios para hacerlo.

Dentro de las diversas clases de objetos **View** que usarás para componer tu diseño, quizá observes varios métodos de retrollamada (*callback*) públicos que pueden ser útiles para eventos de IU. La infraestructura (*framework*) de Android llama a estos métodos cuando la acción respectiva ocurre en el objeto vista. Por ejemplo, cuando el usuario interacciona con un objeto View (por ejemplo, un botón), se produce una llamada al método **onTouchEvent()** en ese objeto. Sin embargo, para interceptar esto, debes extender la clase y sobrescribir el método. No obstante, extender todas las subclases vista para manejar tal evento no sería práctico. Es por ello que la clase de vista también contiene una colección de interfaces anidadas con retrollamadas (*callbacks*) que puedes definir más fácilmente. Estas interfaces, llamadas [receptores de eventos](#)  (*event listeners*), te permiten capturar la interacción del usuario con tu IU.

Si bien generalmente usarás los receptores de eventos para escuchar la interacción del usuario, es posible que en algún momento desees ampliar una clase View para crear un componente personalizado. Quizá desees ampliar la clase [Button](#)  para crear algo más elaborado. En este caso, podrás definir los comportamientos de eventos predeterminados para tu clase utilizando los [controladores de eventos](#)  (*event handlers*) de la clase.

3.2.1.- Receptores de eventos.

Un receptor de eventos (*event listener*) es objeto de una clase que implementa una interfaz en la clase [View](#)  la cual contiene un solo método de retrollamada (*callback*). Estos métodos serán llamados por la infraestructura (*framework*) de Android cuando en la vista en la cual se ha registrado el receptor de eventos se produzca un evento provocado por una interacción del usuario con un elemento de esa vista (en la IU).



[Ministerio de Educación y Formación Profesional](#) (Uso Educativo nc)

En el diagrama podemos ver el modelo de eventos.

En dichas interfaces, se incluyen los siguientes métodos de retrollamada (*callback*):

onClick()

De [View.OnClickListener](#) . Este método es llamado cuando el usuario toca el elemento (en el modo táctil), o selecciona el elemento con las teclas de navegación o la bola de seguimiento y presiona la tecla “Entrar” adecuada o la bola de seguimiento (*trackball*).

onLongClick()

De [View.OnLongClickListener](#) . Este método es llamado cuando el usuario toca y mantiene presionado el elemento (en el modo táctil), o selecciona el elemento con las teclas de navegación o la bola de seguimiento y mantiene presionada la tecla “Entrar” adecuada o la bola de seguimiento (durante un segundo).

onFocusChange()

De [View.OnFocusChangeListener](#) . Este método es llamado cuando el usuario navega hacia el elemento o sale de este utilizando las teclas de navegación o la bola de seguimiento.

onKey()

De [View.OnKeyListener](#) . Este método es llamado cuando el foco de entrada está situado en el elemento y presiona o suelta una tecla física en el dispositivo.

onTouch()

De [View.OnTouchListener](#) . Este método es llamado cuando el usuario realiza una acción calificada como un evento táctil como presionar, soltar o cualquier gesto de

movimiento en la pantalla (dentro de los límites del elemento).

onCreateContextMenu()

De [View.OnCreateContextMenuListener](#) . Este método es llamado cuando se crea un menú contextual (como resultado de un "clic largo" sostenido).

Para más información sobre [menús contextuales](#)  se puede consultar el apartado sobre menús correspondiente en la guía del desarrollador.

Estos métodos son los únicos habitantes de su respectiva interfaz. Para definir uno de estos métodos y manejar sus eventos, implementa la interfaz anidada en tu actividad (*activity*) o defínela como una clase anónima. Luego, pasa una instancia de tu implementación al método **View.set...Listener()** respectivo. (P. ej., llama a [setOnClickListener\(\)](#)  y pásale tu objeto receptor de eventos que debe ser de una clase subtipo de [OnClickListener](#) ).

Ejemplo

A continuación se muestra cómo registrar un receptor de eventos en un botón para responder al evento **onClick**.

```
1 // Creación de un objeto receptor de eventos mediante clase anónima que implement
2 private OnClickListener mCorkyListener = new OnClickListener() {
3     public void onClick(View v) {
4         // Hacer algo cuando el botón es clicado
5     }
6 };
7
8 protected void onCreate(Bundle savedInstanceState) {
9     ...
10    // Obtener el botón buscándolo en nuestra vista
11    Button button = (Button)findViewById(R.id.corky);
12
13    // Registrar el receptor de eventos mCorkyListener en el botón
14    button.setOnClickListener(mCorkyListener);
15    ...
16 }
```

Quizá también te parezca más conveniente implementar **OnClickListener** como parte de tu actividad. Esto evitará la carga extra de la clase y la asignación de objetos.

Ejemplo

```

1 public class ExampleActivity extends Activity implements OnClickListener {
2
3     protected void onCreate(Bundle savedInstanceState) {
4         ...
5         Button button = (Button)findViewById(R.id.corky);
6         button.setOnClickListener(this);
7     }
8
9     // Implementación del método de retrollamada (callback) de la interfaz OnClic
10    public void onClick(View v) {
11        // Hacer algo cuando se clique el botón
12    }
13    ...
14 }

```

Ten en cuenta que el método de retrollamada (*callback*) **onClick()** del ejemplo anterior no tiene valor de retorno, pero algunos otros métodos de receptores de eventos deben devolver un valor lógico (de tipo **boolean**). Cada caso dependerá del tipo de evento. A continuación, se explican los motivos de los pocos casos que lo hacen:

- ✓ **onLongClick()**  : este método devuelve un valor booleano para indicar si has consumido el evento y si no debe continuar. Es decir, devuelve *true* para indicar que has usado el evento y que debe detenerse aquí; devuelve *false* si no has usado el evento, o si el evento debe continuar para otros receptores de clic.
- ✓ **onKey()**  : este método devuelve un valor booleano para indicar si has consumido el evento y si no debe continuar. Es decir, devuelve *true* para indicar que has usado el evento y que debe detenerse aquí; devuelve *false* si no has usado el evento, o si el evento debe continuar para otros receptores de tecla.
- ✓ **onTouch()**  : este método devuelve un valor booleano para indicar si tu receptor consume este evento. Lo importante es que este evento puede tener múltiples acciones una después de la otra. Por lo tanto, si devuelve *false* cuando se recibe el evento de acción de abajo, tú indicas que no has consumido el evento y también que no estás interesado en las acciones subsiguientes de este evento. Por ende, no se te llamará para otras acciones dentro del evento, como un gesto del dedo, o el evento de acción de arriba final.

Nota: Android llamará en primer lugar a los controladores de eventos, y en segundo lugar, a los controladores predeterminados correspondientes de la definición de clase. Por lo tanto, al devolver *true* desde estos receptores de eventos, se detendrá la propagación del evento a otros gestores de eventos y también se bloqueará la retrollamada (*callback*) al controlador de eventos predeterminado en la vista. Por lo tanto, asegúrate de que desees finalizar el evento cuando devuelvas *true*.

Para saber más

Recuerda que los eventos de teclas de hardware siempre se entregan a la vista actualmente en foco. Se distribuyen comenzando desde la parte superior de la jerarquía de vistas y luego hacia abajo, hasta llegar al destino correspondiente. Si tu vista (o un elemento secundario de tu vista) actualmente tiene foco, puedes ver al evento viajar a través del método [dispatchKeyEvent\(\)](#)  . Como alternativa a capturar eventos de teclas a través de tu vista, también puedes recibir todos los eventos dentro de tu actividad con [onKeyDown\(\)](#)  y [onKeyUp\(\)](#)  .

Además, al considerar la entrada de texto para tu aplicación, recuerda que muchos dispositivos solo tienen métodos de entrada de software . No es necesario que tales métodos se basen en teclas; algunos pueden utilizar entrada de voz, escritura a mano, etc. Incluso si un método de entrada presenta una interfaz similar a un teclado, generalmente **no** desencadenará la familia de eventos *onKeyDown()*. Nunca debes construir una IU que requiera que se presionen teclas específicas para controlarla, salvo que desees limitar tu aplicación a dispositivos con un teclado de hardware. En particular, no utilices estos métodos para validar la entrada cuando el usuario presiona la tecla de entrada; en cambio, utiliza acciones como [IME_ACTION_DONE](#)  para señalar al método de entrada cómo tu aplicación espera reaccionar, para que pueda cambiar su IU de forma significativa. Evita los supuestos sobre la forma en la que un método de entrada de software debe funcionar y simplemente confía en él para proporcionar texto ya formateado a tu aplicación.

3.2.2.- Controladores de eventos.

Los controladores/gestores de eventos (*event handlers*) son los métodos de retrollamada (*callback*) que son invocados por los receptores de eventos (*event listeners*) cuando se produce un evento asociado al receptor de eventos que se registró en el componente fuente del evento.

Por ejemplo, `onClick()` es el método de retrollamada que controla/gestiona el evento de clicado en un componente como por ejemplo un botón. Dicho botón es el componente fuente del evento.

Recordemos que los componentes con los cuales puede interactuar el usuario en la interfaz gráfica se denominan controles (*widgets*). En este ejemplo el botón es un control (*widget*).

En el modelo de eventos vimos como el proceso de generación, recepción y gestión de un evento es el siguiente:

1. Registro del receptor de eventos en el componente fuente
2. Interacción del usuario
3. Generación del evento
4. Envío del evento al receptor de eventos (*event listener*)
5. Ejecución del método de retrollamada (*callback*) <----- **Control/gestión del evento**

En el paso 5 se ejecuta alguna acción como reacción al evento que se ha producido en el componente fuente del evento ante la interacción del usuario.

En la siguiente tabla vemos algunos controladores de eventos (métodos de retrollamada) y sus correspondientes interfaces para objetos receptores de eventos.

Controlador de eventos	Receptor de eventos (interfaces)
<code>onClick(View v)</code>	OnClickListener 
<code>onLongClick(View v)</code>	OnLongClickListener() 
<code>onFocusChange(View v, boolean hasFocus)</code>	OnFocusChangeListener() 
<code>onKey(View v, int keyCode, KeyEvent event)</code>	OnKeyListener 
...	...
...	...

Ejercicio Resuelto

Actividad: busca en la API de Android dos controladores de eventos más completando la tabla anterior.

Mostrar retroalimentación

Si buscas en la página de *developer.android.com* por ejemplo con estas palabras clave "api eventos de entrada" el primer resultado te lleva a esta URL [Descripción general de eventos de entrada](#) . Allí puedes encontrar información de los otros 2 objetos de escucha de eventos y sus controladores:

Controlador de eventos	Receptor de eventos (interfaces)
onTouch(View v, MotionEvent event)	OnTouchListener 
onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo)	OnCreateContextMenuListener 

Para saber más

Existen otros métodos que es bueno que conozcas; métodos que no forman parte de la clase de vista, pero que pueden afectar directamente la forma en la que puedes usar los eventos. Por lo tanto, cuando administres eventos más complejos dentro de un diseño, considera estos otros métodos:

- ✓ [Activity.dispatchTouchEvent\(MotionEvent\)](#)  : este método permite que tu actividad (objeto [Activity](#) ) intercepte todos los eventos táctiles antes de que se envíen a la ventana.
- ✓ [ViewGroup.onInterceptTouchEvent\(MotionEvent\)](#)  : este método permite que un objeto [ViewGroup](#)  observe los eventos mientras se envían a las vistas secundarias.
- ✓ [ViewParent.requestDisallowInterceptTouchEvent\(boolean\)](#)  . Llama a este método en una vista primaria para indicar que no debes interceptar eventos táctiles con [onInterceptTouchEvent\(MotionEvent\)](#) .

4.- Ejercicios resueltos.

Caso práctico



[Mikhail \(Pexels\)](#)

Ada ha explicado al equipo todo lo necesario sobre los controles, los eventos y los controladores de eventos, pero es consciente de que hasta que cada uno de los programadores no realice un control de los mismos en una aplicación no van a conseguir interiorizar el funcionamiento.

- Ahora vamos a poner en práctica todo lo que hemos aprendido -les explica Ada-. Para ello os voy a encargar el desarrollo de distintos pequeños proyectos.

- Nos ponemos a ello -responde Juan.

Todo el equipo se ha puesto manos a la obra de forma entusiasmada, compartiendo las soluciones obtenidas.

- Juan, tengo otra solución para el segundo proyecto -dice María-. He encontrado una solución con clases abstractas.

Y es que las posibilidades de la programación orientada a objetos en Java son muy amplias, cada una con sus ventajas e inconvenientes.

4.1.- Proyecto 1. Campo de texto y botón.

Vamos a crear una app que tiene un campo de texto y un botón.

Usaremos el gestor de colocación `LinearLayout` (por ahora sigue los pasos indicados, en otra unidad más adelante estudiaremos los distintos gestores de colocación que podemos usar en Android).

Pasos:

1. Crear un nuevo proyecto en Android Studio usando una actividad vacía como plantilla (no usaremos tildes en el nombre del proyecto para que el nombre del paquete sea generado correctamente)

Nombre: **Proyecto 001. Campo de texto y boton**

Paquete: **com.cidead.pmdm.proyecto001campodetextoyboton**

2. Observamos el diseño por defecto y el contenido XML de **activity_main.xml**. Vemos que disponemos de un control de texto (`TextView`). Este diseño inicial lo vamos a sustituir por el nuestro

3. Sustituimos el contenido de **activity_main.xml** por el que aparece de ejemplo en el apartado 3.C.1:

Código

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="fill_parent"
4      android:layout_height="fill_parent"
5      android:orientation="horizontal">
6
7      <EditText android:id="@+id/edit_message"
8          android:layout_weight="1"
9          android:layout_width="0dp"
10         android:layout_height="wrap_content"
11         android:hint="@string/edit_message" />
12
13     <Button android:id="@+id/button_send"
14         android:layout_width="wrap_content"
15         android:layout_height="wrap_content"
16         android:text="@string/button_send"
17         android:onClick="sendMessage" />
18
19 </LinearLayout>
```

Observamos que en el elemento XML llamado *EditText* que corresponde al campo de texto, aparece en rojo el valor del atributo *android:hint*.

Asimismo, en el elemento XML llamado *Button* que corresponde al botón, aparece en rojo el valor del atributo *android:text*

@string/button_send

Están marcados en rojo porque Android Studio está inspeccionando el código en tiempo de edición y ha detectado esos 2 errores como símbolos desconocidos; y es cierto, ya que no hemos definido esas cadenas (*string*) aún.

Si posamos el cursor del ratón (sin clicar) sobre alguno de esos errores obtenemos información sobre dicho error.

En este primer ejemplo, solucionaremos esos error en los siguientes pasos sin usar por ahora símbolos *@string*.

4. Observamos el diseño + boceto de nuestra app.

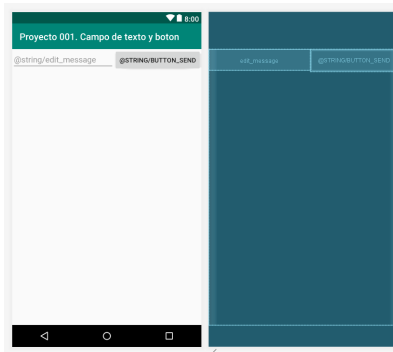
En la esquina superior izquierda pulsando en el icono  podemos determinar que se muestre:

Diseño (*Design*)

Boceto (*Blueprint*)

Diseño+Boceto (*Design+BluePrint*) ← Por defecto

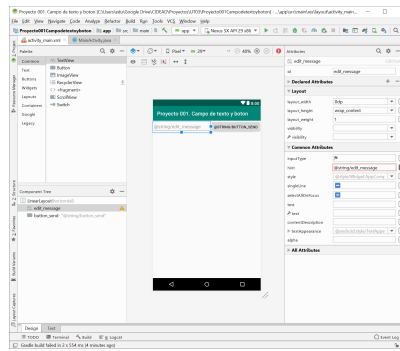
Vamos a elegir la opción de mostrar sólo el diseño (sin boceto).



[Google Developers](https://developer.android.com) (Uso educativo no)

5. En el modo diseño, seleccionamos el campo de texto y observamos las ventanas siguientes:

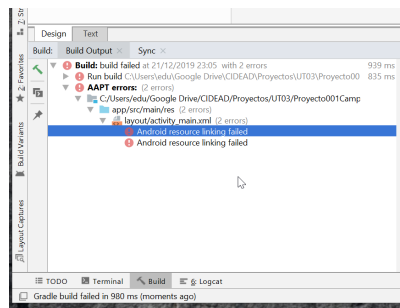
- ✓ *Palette*: paleta de componentes que podemos usar en nuestra app
- ✓ *Component Tree*: árbol de componentes
- ✓ *Attributes*: atributos del componente seleccionado



[Google Developers](#) (Uso educativo nc)

6. Pulsamos el icono  para ejecutar nuestra app

Observamos que el resultado de la construcción (*build*) tiene 2 errores en **activity_mail.xml**



[Google Developers](#) (Uso educativo nc)

Así mismo vemos en la ventana de diseño que aparece arriba a la derecha un icono  que indica que existen errores en nuestra app. Si lo pulsamos se despliega la ventana con los errores y advertencias (Warnings) con mensajes e iconos como este  que indica una advertencia.

7. Con el campo de texto seleccionado, en la ventana de atributos, dentro del grupo de atributos comunes (*common attributes*) sustituimos el valor del atributo **hint** por el texto **Escribe tu mensaje**.

Observamos en el modo código (seleccionamos *Code*) como se ha actualizado el valor del atributo `android:hint` en el elemento XML *EditText* y ya no aparece en rojo (un error menos ;-))

Ejecutamos y vemos que solo hay un error ahora. ¡Vamos a arreglarlo!

8. En modo diseño seleccionamos el botón. Observamos que el valor del atributo **text** es **@string/button_send**

En la ventana de salida de la construcción **Build Output** hacemos doble click en el error que queda y nos llevará al modo texto y al elemento **Button** del fichero **activity_main.xml** donde está el error. En este caso vamos a modificar directamente el contenido del fichero XML en vez de usar la ventana de atributos del modo diseño.

Sustituimos el valor **@string/button_send** del atributo `android:text` por el texto **Enviar**

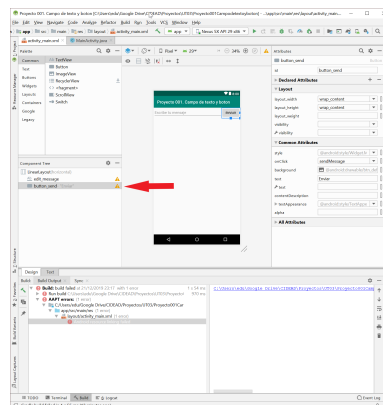
Cuidado: si estás usando la misma API que yo (29) el texto del botón aparece en el modo diseño (y por tanto luego en la ejecución) en mayúsculas a pesar de haberlo escrito con la primera letra mayúsculas y el resto minúsculas. Esto se debe a que en esta versión de la API (y en otras también, aunque no en todas necesariamente) hay un atributo llamado `textAllCaps` que tiene el valor `true` por defecto. Vamos a cambiarlo a `false` para que se muestre el texto tal y como lo hemos escrito.

En el modo de Diseño, en la ventana de atributos, dentro del grupo de todos los atributos, asignamos `false` al atributo `textAllCaps`. Automáticamente dicho atributo y su valor quedan añadidos a:

1. El elemento `Button` de `activity_main.xml` y lo podemos ver en el modo texto.
2. El grupo `Declared Attributes` en el modo diseño; en el cual aparecen exactamente los atributos que hemos declarado en el fichero XML `activity_main.xml` correspondiente.

Este cambio lo hemos hecho como ejemplo pero es posible que en una app "real" queramos que el texto aparezca acorde a la configuración visual por defecto de la API que se use al ejecutar la app.

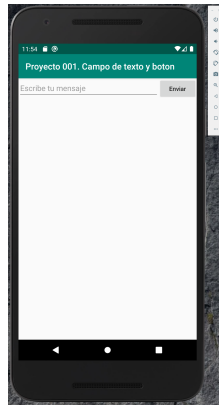
Volvemos al modo diseño y observamos como el valor del atributo **text** ha cambiado a **Enviar** pero tenemos un problemilla... hay una advertencia en la ventana árbol de componentes en el componente **button_send** que antes no teníamos.



[Google Developers](https://developers.google.com/android) (Uso educativo nc)

Ya no tenemos ningún error en nuestra app, aunque tenemos advertencias que solucionaremos más tarde.

9. Ejecutamos  nuestra app.



[Google Developers](#) (Uso educativo nc)

Escribimos un mensaje en el campo de texto y pulsamos el botón **Enviar**.

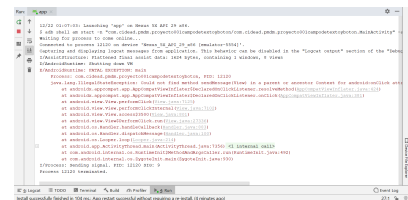
10) Nuestra app termina su ejecución aparentemente sin errores pero... no es así.

Observemos la ventana **Run** (Alt+4). Vemos una línea con el comando adb:

\$ adb shell ...

Para saber más

1. Buscar qué es y para qué sirve el programa adb.
2. Buscar en tu ordenador en qué directorio está el programa adb.exe (sólo si trabajas con Windows)
3. Observar el error en la ventana Logcat (Alt+6) en vez de en la ventana Run (Alt+4)



[Google Developers](#) (Uso educativo nc)

Nos centramos ahora en ver el error. Tras pulsar el botón se ha producido una excepción: **java.lang.IllegalStateException** con el mensaje:

Could not find method sendMessage(View) in a parent or ancestor Context for android:onClick attribute defined on view class androidx.appcompat.widget.AppCompatButton with id 'button_send'

Al pulsar el botón cuyo identificador es **button_send** se ha producido un evento y Android ha intentado invocar al método **sendMessage(View)** tal y como lo hemos definido en el fichero **activity_main.xml** para gestionar el evento que se ha producido.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="fill_parent"
4     android:layout_height="fill_parent"
5     android:orientation="horizontal">
6
7     <EditText
8         android:id="@+id/edit_message"
9         android:layout_width="0dp"
10        android:layout_height="wrap_content"
11        android:layout_weight="1"
12        android:hint="Escribe tu mensaje" />
13
14    <Button
15        android:id="@+id/button_send"
16        android:layout_width="wrap_content"
17        android:layout_height="wrap_content"
18        android:onClick="sendMessage"
19        android:text="Enviar"
20        android:textAllCaps="false" />
21
22 </LinearLayout>

```

[Google Developers](#) (Uso educativo nc)

Pero... ese método no existe. Por tanto se produce la excepción y el programa termina.

11. Para solucionar el error debemos crear ese método **sendMessage(View)** en la clase MainActivity. Pero vamos a cambiarle el nombre para ponerlo en español. El nuevo método **enviarMensaje(View)** será por tanto el controlador/gestor del evento de clicado en el botón.

Sustituimos el valor de **android:onClick** por **enviarMensaje** en **activity_main.xml**.

```

<Button
    android:id="@+id/button_send"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:onClick="enviarMensaje"
    android:text="Enviar"
    android:textAllCaps="false" />

```

[Google Developers](#) (Uso educativo nc)

El atributo **android:text** aparece marcado con fondo amarillo ya que su valor tiene una advertencia por no usar **@string** (las advertencias las solucionaremos más adelante)

Creamos el método **enviarMensaje(View)** en la clase **MainActivity**.

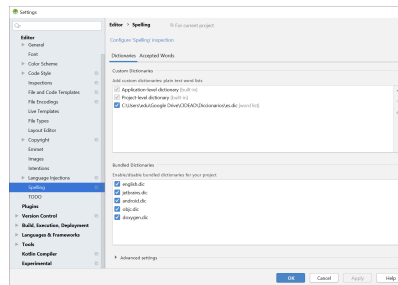
```

1 package com.cidead.pmdm.proyecto001campodetextoyboton;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7
8 public class MainActivity extends AppCompatActivity {
9
10    @Override
11    protected void onCreate(Bundle savedInstanceState) {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.activity_main);
14    }
15
16    public void enviarMensaje(View v) {
17        System.out.println("Se ha pulsado un botón");
18    }
19 }

```

[Google Developers](#) (Uso educativo nc)

Para evitar el marcado de palabras no reconocidas por el analizador léxico de idiomas de Android Studio podemos añadir un diccionario personal de nuestro idioma. En nuestro caso usaremos el diccionario español. Podemos descargar un fichero diccionario español (y de otros idiomas) gratis de <http://www.winedt.org/dictASCII.html> . Lo añadimos a Android Studio como diccionario personal y así evitamos las marcas de subrayado ondulado verdes ya que las palabras en español serán reconocidas (por defecto, Android Studio sólo entiende el idioma inglés).



[Google Developers](#) (Uso educativo nc)

Como podemos observar en la siguiente imagen ya no está marcado en nombre del método ni la palabra pulsado pero sí botón (este diccionario no tiene palabras con tilde, habría que buscar otro mejor...). Ya está mejor en cualquier caso...

```

1 package com.cidead.pmdm.proyecto001campodetextoyboton;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7
8 public class MainActivity extends AppCompatActivity {
9
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_main);
14     }
15
16     public void enviarMensaje(View v) {
17         System.out.println("Se ha pulsado un botón");
18     }
19 }

```

[Google Developers](#) (Uso educativo nc)

Ahora si ejecutamos nuestra app veremos el mensaje "Se ha pulsado un botón" en la ventana **Run** cada vez que cliquemos en el botón.

En los próximos ejemplos veremos:

- ✓ Cómo solucionar los problemas indicados por las advertencias en nuestro ejemplo.
- ✓ Cómo usar símbolos @string en vez de literales de cadena en activity_main.xml y las ventajas que ello conlleva.
- ✓ Cómo realizar el mismo ejemplo pero creando el objeto receptor de eventos de manera programática. Veremos si es posible hacerlo de 3 formas distintas:
 - ➡ La misma clase
 - ➡ Clase interna
 - ➡ Clase anónima

Para saber más

Puedes llevar a cabo estas actividades opcionales:

1. Modificar el método enviarMensaje para que muestre el texto introducido en el campo de texto en vez de "Se ha pulsado un botón".
2. Modificar el nombre de los identificadores del campo de texto y del botón para ponerlos en español; y comprobar que la app sigue ejecutándose correctamente.

`edit_message` → `et_mensaje`

`button_send` → `b_enviar`

Documentación sobre botones y eventos:

<https://developer.android.com/guide/topics/ui/controls/button#java> 

Acceso a recursos usando la clase R:

<https://developer.android.com/guide/topics/resources/accessing-resources.html?hl=Es-419> 

4.2.- Proyecto 2. Campo de texto y botón usando @string. Misma clase.

Suponemos que hemos realizado el Proyecto 1. Campo de texto y botón.

Vamos a realizar un **NUEVO** proyecto muy parecido al anterior pero en este caso vemos:

- ✓ Cómo solucionar los problemas indicados por las advertencias en nuestro ejemplo.
- ✓ Cómo usar símbolos @string en vez de literales de cadena en activity_main.xml y las ventajas que ello conlleva.
- ✓ Cómo realizar el mismo ejemplo pero creando el objeto receptor de eventos de manera programática. Veremos si es posible hacerlo de 3 formas distintas:
 - ➡ La misma clase ← ¿Ya lo hicimos en el proyecto 1?
 - ➡ Clase interna ← Proyecto 3
 - ➡ Clase anónima ← Proyecto 4

Al final de este apartado tienes la documentación relacionada con el programa realizado.

Pasos

1.- Crear un nuevo proyecto en Android Studio usando una actividad vacía como plantilla (no usaremos tildes en el nombre del proyecto para que el nombre del paquete sea generado correctamente)

Nombre: **Proyecto 002. Campo de texto y boton usando @string**

Paquete: **com.cidead.pmdm.proyecto002campodetextoybotonusandostring**

2. Observamos el diseño por defecto y el contenido XML de **activity_main.xml**. Vemos que disponemos de un control de texto (TextView). Este diseño inicial lo vamos a sustituir por el nuestro

3. Sustituimos el contenido de **activity_main.xml** por el que aparece de ejemplo en el apartado 3.C.1:

Código

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="fill_parent"
4     android:layout_height="fill_parent"
5     android:orientation="horizontal">
6
7     <EditText android:id="@+id/edit_message"
8         android:layout_weight="1"
9         android:layout_width="0dp"
10        android:layout_height="wrap_content"
11        android:hint="@string/edit_message" />
```

```

12
13     <Button android:id="@+id/button_send"
14             android:layout_width="wrap_content"
15             android:layout_height="wrap_content"
16             android:text="@string/button_send"
17             android:onClick="sendMessage" />
18
19 </LinearLayout>

```

Observamos que en el elemento XML llamado EditText que corresponde al campo de texto, aparece en rojo el valor del atributo **android:hint**

@string/edit_message

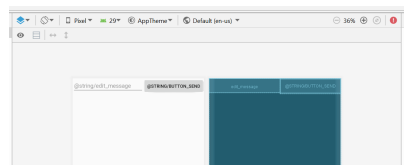
Asimismo, en el elemento XML llamado Button que corresponde al botón, aparece en rojo el valor del atributo **android:text**

@string/button_send

Están marcados en rojo porque Android Studio está inspeccionando el código en tiempo de edición y ha detectado esos 2 errores como símbolos desconocidos; y es cierto, ya que no hemos definido esas cadenas (*string*) aún.

Si posamos el cursor del ratón (sin clicar) sobre alguno de esos errores obtenemos información sobre dicho error.

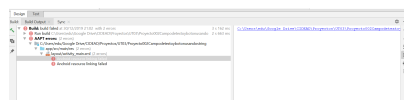
4. Observamos el diseño + boceto de nuestra app. Nos fijamos dónde aparecen dichos valores @string.



[Google Developers](#) (Uso educativo nc)

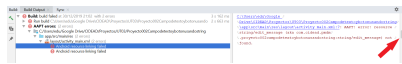
5. Pulsamos el icono  para ejecutar nuestra app

Observamos que el resultado de la construcción (*build*) tiene 2 errores en **activity_mail.xml**



[Google Developers](#) (Uso educativo nc)

Si clicamos en el primero vemos a la derecha la descripción del error; incompleta ya que no cabe horizontalmente en ese marco. Para ver la descripción completa sin tener que redimensionar la ventana de Android Studio, clicamos en el icono Soft-wrap (ajuste suave).



[Google Developers](#) (Uso educativo nc)

El error indica que no ha sido posible encontrar ese recurso.

Si clicamos en el segundo error veremos algo parecido.

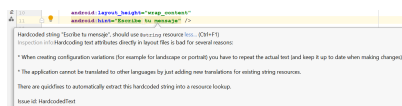
En los siguientes apartados procedemos a solucionarlo.

6. Antes de ello vamos a renombrar refactorizando el identificador del campo de texto y del botón.

Pulsamos Shift+F6 sobre el identificador "@+id/edit_message" y lo sustituimos por "@+id/et_mensaje" **[Refactor]**

Idem "@+id/button_send" a "b_enviar" **[Refactor]**

7. En el modo texto sustituimos "@string/edit_message" por "Escribe tu mensaje". Hacemos esto únicamente con la intención de que Android Studio nos muestre las advertencias que nos indican porqué esta sustitución por literales de cadena no es buena idea...



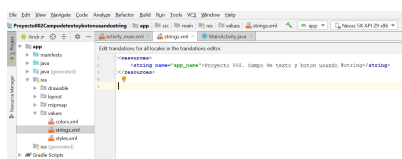
[Google Developers](#) (Uso educativo nc)

Dos desventajas:

- Si creamos 2 diseños distintos (por ejemplo: vertical y horizontal) tendremos que repetir el texto y mantenerlo actualizado en las dos versiones de colocación (en los ficheros XML correspondientes o en código)
- La app no puede ser traducida a otros idiomas añadiendo nuevas traducciones para recursos de cadena (*string*) existentes. Por tanto, complicamos el esfuerzo para la internacionalización de nuestra app

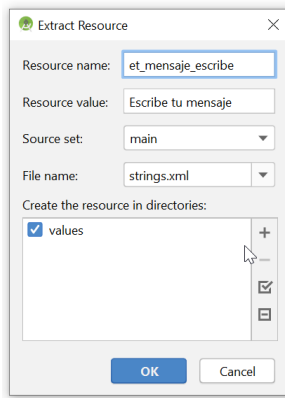
Dejaremos que Android Studio nos ayude a crear los recursos de cadena correspondientes.

8. Observamos el fichero **strings.xml** en el cual se guardan los recursos de cadena de la app. Al crear el proyecto tenemos el nombre de la app como recurso de cadena definido automáticamente por Android Studio.



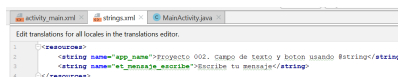
[Google Developers](#) (Uso educativo nc)

9. Vamos a añadir nuestros recursos de cadena desde **activity_main.xml**. Pulsamos Alt+Enter sobre el texto "Escribe tu mensaje" (ó pulsamos sobre la bombilla) y elegimos la opción **Extract string resource** y le ponemos de nombre al recurso: **et_mensaje_escribe**



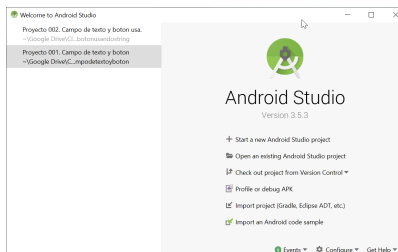
[Google Developers](#) (Uso educativo nc)

10. Observamos como se ha añadido el recurso de cadena et_mensaje_escribe a **strings.xml**



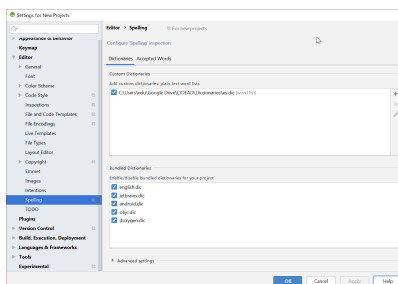
[Google Developers](#) (Uso educativo nc)

Vemos como Android Studio está marcando con líneas verdes onduladas los errores léxicos idiomáticos a pesar de que en el proyecto anterior añadimos un diccionario personalizado de español. Lo añadimos... sí... pero lo hicimos sólo para el proyecto actual, es decir, para el proyecto 1. Si queremos que ese diccionario esté disponible para todos los nuevos proyectos lo tendremos que añadir en la ventana inicial de Android Studio:



[Google Developers](#) (Uso educativo nc)

Configure | Settings | Editor | Spelling



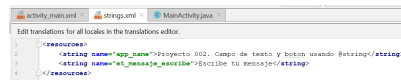
[Google Developers](#) (Uso educativo nc)

Pero esto sólo funcionará para los nuevos proyectos creados a partir de ahora... para que funcione en nuestro proyecto actual, debemos añadir el diccionario personalizado teniendo

abierto nuestro Proyecto 002 en:

File | Settings (Ctrl+Alt+S) | Editor | Spelling | Dictionaries | Custom Dictionaries

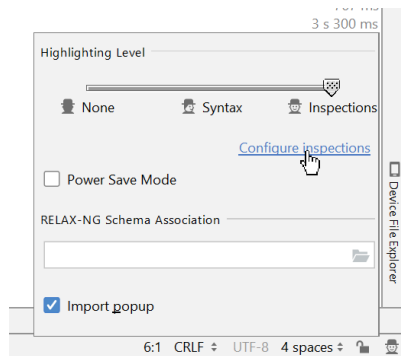
Y ya no veremos esos errores léxicos idiomáticos (líneas verdes onduladas):



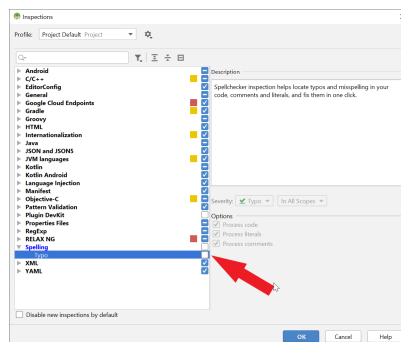
[Google Developers](#) (Uso educativo nc)

El error léxico en la palabra **boton** es porque falta la tilde que no pusimos adrede al crear el proyecto para evitar determinados problemas con la codificación en sistemas operativos y programas que usen normas de codificación que no incluyan caracteres específicos del idioma español como la tilde.

Si no quisiéramos ver ningún error idiomático (ni siquiera el de la tilde) en nuestro proyecto podemos desactivarlo de la siguiente forma:

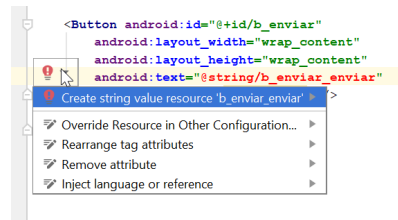


[Google Developers](#) (Uso educativo nc)



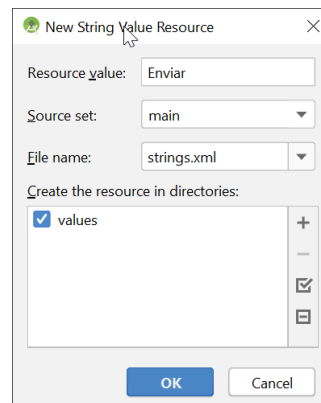
[Google Developers](#) (Uso educativo nc)

11. Vamos a crear el recurso de cadena para el texto del botón al igual que lo hicimos anteriormente para el atributo **hint** del campo de texto. En este caso el procedimiento no es el mismo. Renombramos "@string/button_send" a "@string/b_enviar_enviar". Pulsamos Alt+Enter y elegimos **Create string value resource**.



[Google Developers](#) (Uso educativo nc)

Le asignamos como valor del recurso: **Enviar**



[Google Developers](#) (Uso educativo nc)

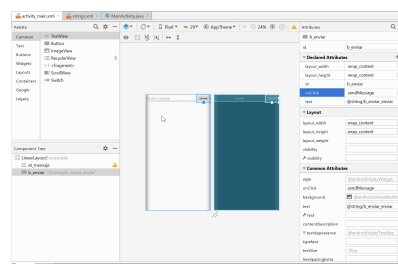
12. Observamos **strings.xml**



[Google Developers](#) (Uso educativo nc)

13. Pulsamos el icono  para ejecutar nuestra app y deberíamos tener éxito.

Podríamos haber creado los recursos de cadena escribiéndolos "a mano" en **strings.xml** y seleccionándolos desde el modo diseño en la ventana de atributos. Ver siguiente imagen (observar como tienen óvalos en negro a la derecha los atributos con recursos asociados a su valor).



[Google Developers](#) (Uso educativo nc)

Actividad: clicas en algún óvalo (*Pick a resource*) de algún atributo y observar como aparecen recursos seleccionables propios y de Android.

14. Si nos fijamos en **activity_main.xml** vemos que Android Studio marca con fondo amarillo **sendMessage** y nos informa que no puede encontrar ese método y... es cierto.



[Google Developers](#) (Uso educativo nc)

No existe y provocará un error de ejecución al clicar en el botón.

Actividad: ejecutar y comprobar el error de ejecución que se produce al pulsar el botón por no existir tal método **sendMessage**.

15. Resolvemos AHORA la parte de crear los **objetos receptores de eventos** de forma **programática**.

Dijimos al principio de este proyecto: "Cómo realizar el mismo ejemplo pero creando el objeto receptor de eventos de manera programática. Veremos si es posible hacerlo de 3 formas distintas:

- ✓ La misma clase ← ¿Ya lo hicimos en el proyecto 1?
- ✓ Clase interna
- ✓ Clase anónima"

Realmente no hemos creado programáticamente un objeto receptor de eventos ni lo hemos registrado explícitamente en el componente origen del evento (el botón). Lo que hicimos en el proyecto 1 fue especificar cuál es el método de retrollamada asociado al evento de clicado del botón y escribir el código correspondiente (en Java) del método de retrollamada **enviarMensaje** en la clase **MainActivity**.

Del proyecto 1 (**activity_main.xml**)

```
<Button
    android:id="@+id/button_send"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:onClick="enviarMensaje"
    android:text="Enviar"
    android:textAllCaps="false" />
```

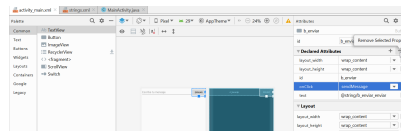
[Google Developers](#) (Uso educativo nc)

Del proyecto 1 (**MainActivity.java**)

```
1 package com.cidead.pmdm.proyecto001campodetextoyboton;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7
8 public class MainActivity extends AppCompatActivity {
9
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_main);
14     }
15
16     public void enviarMensaje(View v) {
17         System.out.println("Se ha pulsado un botón");
18     }
19 }
```

[Google Developers](#) (Uso educativo nc)

En este proyecto vamos a crear el objeto receptor de eventos con su método de retrollamada de forma programática (escribiendo código en Java). Por tanto, eliminamos el atributo **onClick** (y su valor) del elemento **Button**. Lo podemos hacer editando el fichero **activity_main.xml** directamente o bien en modo diseño. Esta vez lo haremos en modo diseño (teniendo seleccionado el botón **b_enviar**) pulsando el signo - sobre el atributo **onClick** de los atributos declarados (*declared attributes*). Observamos como el atributo también desaparece (lógicamente) del fichero XML **activity_main.xml**.



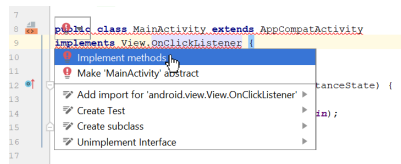
[Google Developers](#) (Uso educativo nc)

```
<Button
    android:id="@+id/b_enviar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/b_enviar_enviar" />
```

[Google Developers](#) (Uso educativo nc)

16. La MISMA CLASE como objeto receptor de eventos

Añadimos la interface **View.OnClickListener** a la definición de la clase MainActivity. Seleccionamos **Implement methods** (clicando en la bombilla roja con admiración o pulsando Alt+Enter) y luego el método **onClick** y escribimos el código de su cuerpo para mostrar un mensaje por la salida estándar de texto como respuesta al evento de clicado del componente origen del evento (que será el botón en nuestro caso).



[Google Developers](#) (Uso educativo nc)

Cuerpo del método de retrollamada **onClick(View)**:

```
System.out.println("Se ha pulsado un botón");
```

```
1 package com.cicdead.pmdm.proyecto002campodetextoybotonusaandostring;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.view.View;
6
7 public class MainActivity extends AppCompatActivity
8 implements View.OnClickListener {
9
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_main);
14     }
15
16     @Override
17     public void onClick(View v) {
18         System.out.println("Se ha pulsado un botón");
19     }
20 }
21
```

[Google Developers](#) (Uso educativo nc)

Si ejecutamos el programa vemos que al pulsar el botón no ocurre nada. Piensa un momento

Google Developers (Uso educativo nc)



Dejaremos los casos restantes (clase interna y anónima) para los proyectos 3 y 4 (respectivamente).

A continuación te proponemos unos enlaces que te permitirán ampliar información sobre estas cuestiones:

1. [Documentación sobre botones y eventos](#) 
2. [Acceso a recursos usando la clase R](#) 
3. [Descripción general de los eventos de entrada](#) 



4.3.- Proyecto 3. Campo de texto y botón usando @string. Clase interna.

Usaremos en ese caso una clase interna para crear el objeto receptor de eventos del botón. Partimos de una copia del proyecto 2.

PASOS PARA DUPLICAR UN PROYECTO EXISTENTE (en este caso el Proyecto 2)

1. Cerramos el proyecto anterior (proyecto 2)
2. Comprimos el directorio del proyecto (Proyecto002Campodetextoybotonusandostring) para disponer de una copia de seguridad (.rar por ejemplo)
3. Renombramos el directorio Proyecto002... a Proyecto003Campodetextoybotonusandostring**claseinterna**
4. Ejecutamos Android Studio
5. Abrimos el Proyecto 003
6. En **strings.xml**, modificamos el nombre del proyecto cambiando el valor de la variable **app_name** a Proyecto 003. Campo de texto y boton usando @string. Clase interna
7. En la ventana de Proyecto (Android) | app | manifests | **AndroidManifest.xml** renombramos refactorizando, para lo que seleccionamos el nombre del paquete antiguo (com.cidead...) y con botón derecho seleccionamos Rename o pulsamos Shift+F6. Luego tecleamos el nuevo nombre del paquete (**package**) a **proyecto003campodetextoybotonusandostringclaseinterna** y pulsamos sobre el botón abajo [Do Refactor].
8. En la ventana de Proyecto (Android) | Gradle Scripts | settings.gradle cambiamos el nombre del proyecto de esta forma:
rootProject.name='Proyecto 003. Campo de texto y boton usando @string. Clase interna' y pulsamos arriba a la derecha en la barra azul que ha aparecido **Sync now**
9. Ejecutamos Build | Rebuild project
10. Cerramos el proyecto mediante File | Close Project
11. Ejecutamos Android Studio
12. Abrimos el proyecto usando la opción Open y buscando el proyecto 3
13. Comprobamos que el nombre del proyecto aparece correctamente en el título de la ventana de Android Studio, así como en **strings.xml**, en el fichero de manifiesto **Manifest.xml** y en **settings.gradle** dentro de Gradle Scripts.
14. Si todo es correcto, ejecutamos la app y observamos que el nombre del proyecto salga en la parte superior del móvil del emulador
15. Cerramos el proyecto
16. Descomprimos el proyecto 2 y comprobamos que funciona y tiene asociados nombres de proyecto 002 en los sitios indicados
17. Idem paso anterior con el proyecto 3

PASOS PARA REALIZAR EL PROYECTO 3 USANDO UNA CLASE INTERNA

1. En la clase MainActivity eliminamos la línea: **implements View.OnClickListener**
2. Creamos una clase interna privada (sólo se usará en esta clase en principio) llamada ReceptorBoton con el método de retrollamada que ya teníamos onClick(View)

3. Cambiamos **this** por una instancia de la clase interna que actuará de receptor del evento de clicado del botón **bEnviar**:
bEnviar.setOnClickListener(new ReceptorBoton());
4. Ejecutamos nuestra app.

La ventaja de usar una clase interna (con nombre) respecto de una clase anónima (como veremos en el siguiente proyecto), es que podemos crear tantas instancias como queramos para usarlas como receptores de eventos allá donde lo necesitemos.

```
activity_main.xml | AndroidManifest.xml | MainActivity.java | strings.xml | settings.gradle
1 package com.cleard.pwds.proyecto003campodetextoybotonusandotringclaseInterna;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.Button;
8 import android.widget.EditText;
9
10 public class MainActivity extends AppCompatActivity {
11
12     Button bEnviar;
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17         setContentView(R.layout.activity_main);
18
19         bEnviar=(Button)findViewById(R.id.b_enviar);
20         bEnviar.setOnClickListener(new ReceptorBoton());
21     }
22
23     private class ReceptorBoton implements View.OnClickListener {
24
25         @Override
26         public void onClick(View v) {
27             EditText et1=findViewById(R.id.et_message);
28             System.out.println(et1.getText());
29         }
30     }
```

[Google Developers](#) (Uso educativo nc)

4.4.- Proyecto 4. Campo de texto y botón usando @string. Clase anónima.

Partimos del proyecto 3. Usaremos una clase (interna) anónima como receptor de eventos de nuestro botón.

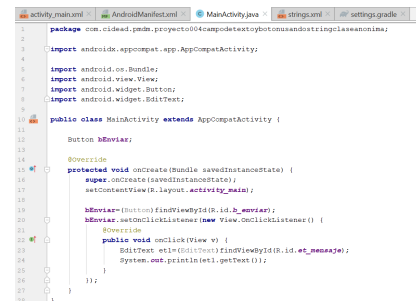
Duplicamos el proyecto 3 en el proyecto 4 siguiendo los pasos aprendidos en el proyecto 3.

Restauramos (extraemos) el proyecto 3 a partir de su archivo comprimido.

PASOS

1. Creamos una instancia de una clase anónima que será un subtipo de `View.OnClickListener` conteniendo el método `onClick` usado en el proyecto anterior.
2. Registramos el objeto receptor de eventos pasando una instancia de esta clase anónima al método `setOnClickListener` del botón.

En el caso de querer reusar esa clase anónima (con sus métodos) no podríamos al no tener nombre, a diferencia de las clases internas (con nombre).



```
1 package com.cidead.pdm.proyecto04campodetextoybotonmandandotextoanónima;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.Button;
8 import android.widget.EditText;
9
10 public class MainActivity extends AppCompatActivity {
11
12     Button bEnviar;
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17         setContentView(R.layout.activity_main);
18
19         bEnviar=(Button)findViewById(R.id.b_sendar);
20         bEnviar.setOnClickListener(new View.OnClickListener() {
21             @Override
22             public void onClick(View v) {
23                 EditText et1=findViewById(R.id.et_mensaje);
24                 System.out.println(et1.getText());
25             }
26         });
27     }
28 }
```

[Google Developers](#) (Uso educativo no)

4.5.- Proyecto 5. Texto, imagen y formato.

Vamos a crear una app que mostrará un texto (objeto **TextView**) de color rojo, tamaño 30dp y estilo de texto cursiva. El texto se mostrará centrado horizontal y verticalmente. Además el fondo de la app tendrá un color.

Usaremos el gestor de colocación RelativeLayout (por ahora sigue los pasos indicados, en otra unidad más adelante estudiaremos los distintos gestores de colocación que podemos usar en Android)

1. Crear un nuevo proyecto en Android Studio usando una actividad vacía como plantilla (no usaremos tildes en el nombre del proyecto para que el nombre del paquete sea generado correctamente)

Nombre: **Proyecto 005. Texto, imagen y formato**

Paquete: **com.cidead.pmdm.proyecto005textoimagenyformato**

2. Observamos el diseño por defecto y el contenido XML de **activity_main.xml**. Vemos que disponemos de un control de texto (TextView). Este diseño inicial lo vamos a sustituir por el nuestro:

Código

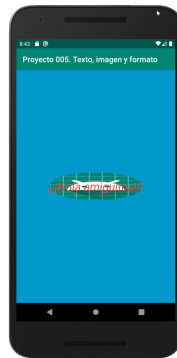
```
1  <?xml version="1.0" encoding="utf-8"?>
2  <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:gravity="center"
8      android:background="@android:color/holo_blue_dark"
9      tools:context=".MainActivity">
10
11     <TextView
12         android:id="@+id/textView"
13         android:layout_width="wrap_content"
14         android:layout_height="wrap_content"
15         android:gravity="center"
16         android:background="@mipmap/ic_launcher"
17         android:shadowColor="@android:color/background_dark"
18         android:text="¡¡¡Hola amiguitos!!!"
19         android:textSize="30dp"
20         android:textColor="@android:color/holo_red_light"
21         android:textStyle="italic"/>
22
23 </RelativeLayout>
```

El fichero MainActivity.java lo dejamos como está:

Código

```
1 package com.cidead.pmdm.proyecto005textoimagenyformato;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        setContentView(R.layout.activity_main);
13    }
14 }
```

Ejecutamos  nuestra app.



[Google Developers](#) (Uso educativo nc)

4. Funciona, pero... hay algunas cosas que no hemos hecho del todo bien...

```
11 <TextView
12     android:id="@+id/textView"
13     android:layout_width="wrap_content"
14     android:layout_height="wrap_content"
15     android:gravity="center"
16     android:background="@mipmap/ic_launcher"
17     android:shadowColor="@android:color/background_dark"
18     android:text="¡¡¡Hola amigos!!!"
19     android:textSize="30dp"
20     android:textColor="@android:color/holo_red_light"
21     android:textStyle="italic"/>
```

[Google Developers](#) (Uso educativo nc)

5. Antes de solucionar esas dos advertencias marcadas en fondo amarillo por Android Studio, nos fijamos en el identificador del elemento **TextView**: se llama textView. Le ponemos

un nombre algo más significativo sobre todo si más adelante añadimos otros controles de texto. Usamos renombrar refactorizando (Shift+F6) sobre el valor "@+id/textView" (en la línea 12 de la imagen) del atributo **id** del elemento **TextView**. Cambiamos **textView** por **tv_saludo**, quedando así: **android:id="@+id/tv_saludo"**

```
11 <TextView
12     android:id="@+id/tv_saludo"
13     android:layout_width="wrap_content"
14     android:layout_height="wrap_content"
15     android:gravity="center"
16     android:background="@mipmap/ic_launcher"
17     android:shadowColor="@android:color/background_dark"
18     android:text="!!!Hola amiguitos!!!"
19     android:textSize="30dp"
20     android:textColor="@android:color/holo_red_light"
21     android:textStyle="italic"/>
```

[Google Developers](#) (Uso educativo nc)

Notación:

- ✓ Usaremos el signo de subrayado para separar palabras en identificadores en XML. En Java usaremos en cambio la notación *Camel/Case* (sin subrayados)
- ✓ Usaremos contracciones para notar los valores de elementos como **tv** para **TextView**

6. Debemos usar **@string** para el atributo **text** del elemento **TextView** (usamos Alt+Enter (ó la bombilla naranja) y **Extract String Resource** sobre la cadena "!!!Hola amiguitos!!!")

```
11 <TextView
12     android:id="@+id/tv_saludo"
13     android:layout_width="wrap_content"
14     android:layout_height="wrap_content"
15     android:gravity="center"
16     android:background="@mipmap/ic_launcher"
17     android:shadowColor="@android:color/background_dark"
18     android:text="@string/tv_saludo_hola"
19     android:textSize="30dp"
20     android:textColor="@android:color/holo_red_light"
21     android:textStyle="italic"/>
```

[Google Developers](#) (Uso educativo nc)

Observamos en la ventana de proyecto (Alt+1) el fichero **app | res | values | strings.xml**

```
activity_main.xml | strings.xml | MainActivity.java
Edit translations for all locales in the translations editor.
1 <resources>
2     <string name="app_name">Proyecto 005. Texto, imagen y formato</string>
3     <string name="tv_saludo_hola">!!!Hola amiguitos!!!</string>
4 </resources>
```

[Google Developers](#) (Uso educativo nc)

7. Resolvemos ahora la segunda advertencia sobre el atributo **textSize**.

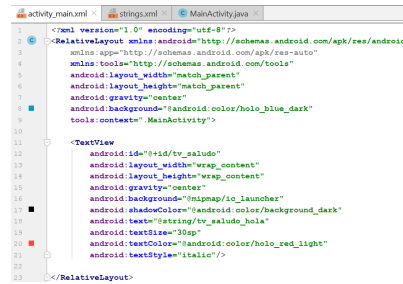


[Google Developers](#) (Uso educativo nc)

Por defecto el tamaño del texto está en **dp**. Sin embargo, Android Studio nos indica que debemos usar la unidad **sp** en vez de **dp** para tamaños de texto.

Si usamos **sp** Android tendrá en cuenta las preferencias del usuario sobre tamaño de texto así como la densidad de la pantalla. Hay casos en los que será mejor usar **dp**; por ejemplo, cuando el texto está en un contenedor con un tamaño específico especificado en unidad **dp**. Esto evitará que el texto pueda "salirse" fuera del contenedor si el usuario ha configurado un tamaño de texto personalizado mayor. Pero el hecho de usar la unidad **dp** implica no respetar las preferencias de tamaño de texto del usuario; por tanto, la buena práctica será definir un diseño que sea flexible/adaptable (*responsive*).

Por tanto, cambiamos la unidad **dp** por **sp** en el valor del atributo **textSize**.

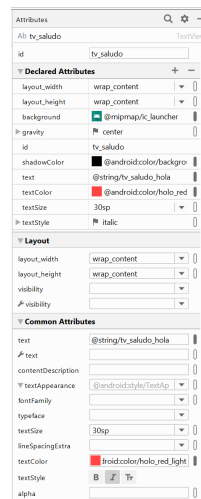


```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     android:gravity="center"
8     android:background="@android:color/hoio_blue_dark"
9     tools:context=".MainActivity">
10
11     <TextView
12         android:id="@+id/tv_saludo"
13         android:layout_width="wrap_content"
14         android:layout_height="wrap_content"
15         android:gravity="center"
16         android:background="@mipmap/ic_launcher"
17         android:shadowColor="@android:color/background_dark"
18         android:text="@string/tv_saludo_hola"
19         android:textSize="30sp"
20         android:textColor="@android:color/hoio_red_light"
21         android:textStyle="italic"/>
22
23 </RelativeLayout>
```

[Google Developers](#) (Uso educativo nc)

8. Ejecutamos  nuestra app y comprobamos que todo va bien.

9. Antes de terminar con este proyecto dedica un tiempo a observar los atributos y sus valores del elemento **TextView tv_saludo** en modo diseño.



[Google Developers](#) (Uso educativo nc)

10. Puedes intentar cambiar el valor de alguno de ellos para volver a ejecutar la app y observar los cambios.

Para saber más

[Definición de píxel independiente del dispositivo dp en wikipedia.](#) 

[Guía práctica sobre el uso de pantallas en Android studio.](#) 

4.6.- Proyecto 6. Saludo con botón.

Vamos a crear una app que mostrará un texto (objeto **TextView**) de color azul, tamaño de texto 20sp y centrado, cuando se pulse el botón **Saludar**. Para hacer este ejercicio usaremos el documento XML para especificar el método de retrollamada **saludar** el cual será invocado al clicarse el botón.

Apariencia:



[Google Developers](#) (Uso educativo nc)



[Google Developers](#) (Uso educativo nc)

En este caso, nuestra aplicación ya tendrá, aunque de manera muy básica, cierta interacción con el usuario a través de la gestión de evento de clicado sobre un botón. El registro de dicho evento lo haremos a través de XML, de forma que en el elemento **Button** incorporaremos un atributo "**android:onClick**". No es quizá la forma más elegante, pero sí es completamente funcional, y sobre todo, muy sencilla y apropiada en los primeros pasos en Android.

Además, usaremos el fichero **strings.xml** para incorporar las cadenas de texto que va a utilizar mi aplicación, evitando escribirlas directamente o "en bruto" sobre el propio **activity_main.xml**. Esta manera de proceder es la más correcta y la que debemos usar de aquí en adelante.

Por último, decir que en el fichero **MainActivity.java** desarrollaremos el método de retrollamada **saludar**. Dicho método es sencillo; únicamente juega con la visibilidad del

control de texto (**TextView**). Inicialmente es invisible. Definido así en **activity_main.xml**:

`android:visibility="invisible"`

y al ser clicado se hace visible

```
tvSaludo.setVisibility(View.VISIBLE);
```

strings.xml (en carpeta "values" dentro de carpeta de recursos "res")

```
1 <resources>
2   <string name="app_name">Proyecto 006. Saludo con boton</string>
3   <string name="b_saludar_saludar">Saludar</string>
4   <string name="tv_saludo_hola">Hola amig@s, bienvenid@s al curso de Android St
5 </resources>
```

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3   xmlns:tools="http://schemas.android.com/tools"
4   android:layout_width="match_parent"
5   android:layout_height="match_parent"
6   tools:context=".MainActivity">
7
8   <Button
9     android:id="@+id/b_saludar"
10    android:layout_width="wrap_content"
11    android:layout_height="wrap_content"
12    android:text="@string/b_saludar_saludar"
13    android:layout_centerHorizontal="true"
14    android:onClick="saludar"/>
15
16   <TextView
17     android:id="@+id/tv_saludo"
18     android:layout_width="wrap_content"
19     android:layout_height="wrap_content"
20     android:layout_below="@id/b_saludar"
21     android:layout_centerHorizontal="true"
22     android:text="@string/tv_saludo_hola"
23     android:textAlignment="center"
24     android:textColor="#3F51B5"
25     android:textSize="20sp"
```

```
26         android:visibility="invisible" />
27     </RelativeLayout>
```

MainActivity.java

```
1  package com.cidead.pmdm.proyecto006saludoconboton;
2
3  import androidx.appcompat.app.AppCompatActivity;
4
5  import android.os.Bundle;
6  import android.view.View;
7  import android.widget.TextView;
8
9  public class MainActivity extends AppCompatActivity {
10
11     TextView tvSaludo;
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         setContentView(R.layout.activity_main);
17         tvSaludo=(TextView)findViewById(R.id.tv_saludo);
18     }
19
20     public void saludar(View view) {
21         tvSaludo.setVisibility(View.VISIBLE);
22     }
23 }
```

4.7.- Proyecto 7. Saludo con botón. Clase anónima.

Vamos a crear una app igual que la anterior pero en este caso incorporaremos control del evento de clicado en el propia clase **MainActivity**, es decir, haremos uso de una definición programática del evento para resolver el ejercicio. Usaremos para crear el receptor del evento una clase anónima. La apariencia será la misma que la del ejercicio anterior.

Apariencia:



[Google Developers](#) (Uso educativo nc)



[Google Developers](#) (Uso educativo nc)

En este caso, vamos a tratar de resolver el ejercicio anterior de forma que la parte de programación sea más independiente de la parte de diseño, es decir, por una parte Java, y por otra parte XML. Ahora quiero que el registro del evento se realice en el fichero Java (**MainActivity.java**). Por eso, ya no tendremos el atributo **android:onClick** en el elemento **Button** en el fichero **activity_main.xml**. Sin embargo, ahora en **MainActivity.java**, usaremos el método **setOnClickListener** para el registro del receptor de eventos en el botón.

No debes asustarte por el tamaño de la estructura, de hecho lo interesante es aprovechar las funciones de autocompletado que te brinda el entorno de desarrollo integrado Android Studio, de forma que autogenera, con muy poco código escrito por ti, el código que falte.

strings.xml

```
1 <resources>
2     <string name="app_name">Proyecto 007. Saludo con boton. Clase anonima</string>
3     <string name="b_saludar_saludar">Saludar</string>
4     <string name="tv_saludo_hola">Hola amig@s, bienvenid@s al curso de Android St
5 </resources>
```



activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <Button
10         android:id="@+id/b_saludar"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="@string/b_saludar_saludar"
14         android:layout_centerHorizontal="true"/>
15
16     <TextView
17         android:id="@+id/tv_saludo"
18         android:layout_width="wrap_content"
19         android:layout_height="wrap_content"
20         android:textAlignment="center"
21         android:layout_below="@id/b_saludar"
22         android:layout_centerHorizontal="true"/>
23 </RelativeLayout>
```

MainActivity.java

```

1 package com.cidead.pmdm.proyecto007saludoconbotonclaseanonima;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.graphics.Color;
6 import android.os.Bundle;
7 import android.view.View;
8 import android.widget.Button;
9 import android.widget.TextView;
10
11 public class MainActivity extends AppCompatActivity {
12     Button bSaludar;
13     TextView tvSaludo;
14
15     @Override
16     protected void onCreate(Bundle savedInstanceState) {
17         super.onCreate(savedInstanceState);
18         setContentView(R.layout.activity_main);
19
20         // Inicialización de controles (botón y texto)
21         bSaludar=(Button)findViewById(R.id.b_saludar);
22         tvSaludo=(TextView)findViewById(R.id.tv_saludo);
23
24         // Registro del receptor de eventos en el botón
25         bSaludar.setOnClickListener(new View.OnClickListener() {
26             @Override
27             public void onClick(View v) {
28                 tvSaludo.setText(getResources().getString(R.string.tv_saludo_hola));
29                 tvSaludo.setTextSize(20);
30                 tvSaludo.setTextColor(Color.parseColor("#3F51B5"));
31             }
32         });
33     }
34 }

```

Observa esta sentencia:

```
tvSaludo.setText(getResources().getString(R.string.tv_saludo_hola));
```

Hemos usado `R.string` para acceder al recurso de cadena de texto **tv_saludo_hola** definido en **strings.xml**

Comprueba que es necesario usar `getResources().getString()` para obtener la cadena del saludo, ya que si usamos directamente `R.string.tv_saludo_hola` obtendremos el identificador numérico de ese recurso de cadena que no es lo que necesitamos.

CONSEJO: Añade un punto de ruptura en esa sentencia (la ejecución llegará a ese punto cada vez que se pulse el botón) y usa el depurador de Android Studio ejecutando paso a paso la app. Usa ventanas de depuración (**watches**) para mostrar el contenido de las variables en cada momento de la ejecución.

En **activity_main.xml** usamos el atributo `tools:context`

```
tools:context=".MainActivity"
```

para definir la clase actividad (clase [Activity](#) ) o la clase fragmento (clase [Fragment](#) ) que ha instanciado el gestor de colocación que se está definiendo. En tiempo de ejecución esto permite a Android asociar la clase (definida en el fichero de código fuente; en nuestro caso **MainActivity.java**) que contiene la implementación del comportamiento con la representación definida por el diseño (en XML; en nuestro caso **activity_main.xml**). Esto se puede utilizarse de muchas formas en tiempo de ejecución.

Más adelante veremos como podemos definir recursos como los colores de la app para usarlos de manera homogénea desde nuestro código en Java.

Por ahora observa el contenido del fichero **colors.xml** en app | res | values.

colors.xml

```
1 | <?xml version="1.0" encoding="utf-8"?>
2 | <resources>
3 |     <color name="colorPrimary">#008577</color>
4 |     <color name="colorPrimaryDark">#00574B</color>
5 |     <color name="colorAccent">#D81B60</color>
6 | </resources>
```

Podríamos (y deberíamos) haber usado uno de esos colores definidos (por ejemplo, el color primario) usando la sentencia:

```
tvSaludo.setTextColor(getResources().getColor(R.color.colorPrimary));
```

Realmente no deberíamos de haber usado el valor de color directamente en el código (se ha hecho así únicamente por motivos didácticos para ejemplificar precisamente las desventajas de hacerlo de esa manera) ya que esto dificulta el mantenimiento de la app al no tener centralizados los colores usados. Por tanto, en el futuro, debemos definir los colores que usamos en nuestra app en el fichero **colors.xml** para poder usarlos y reusarlos, así como modificarlos manteniendo un estilo de colores homogéneo en nuestra app.

Para saber más

Investiga distintas formas de especificar un color y usarlo con el método **setTextColor**.

Por ejemplo puedes ver la [Referencia sobre la clase Color](#)  en Android Developers y en webs de programadores como [StackOverflow](#) .



4.8.- Proyecto 8. Ventana emergente. Clase Toast.

Debes conocer

La clase **Toast** proporciona una manera sencilla de mostrar un mensaje al usuario mediante una pequeña ventana emergente que ocupa el espacio necesario para incorporar el mensaje mientras que la actividad actual permanece visible (por debajo). La ventana emergente desaparece tras un tiempo limitado.



[Google Developers](#) (Uso educativo no)

El aspecto por defecto es el mostrado en la imagen de la derecha.

Código

```
1 Context contexto = getApplicationContext();
2 CharSequence texto = "Mensaje enviado";
3 int duracion = Toast.LENGTH_SHORT;
4 Toast toast = Toast.makeText(contexto, texto, duracion);
5 toast.show();
```

En una sola línea podríamos escribir:

```
Toast.makeText(context, text, duration).show();
```

Si el contexto es la actividad actual usaremos **this**.

```
Toast.makeText(this, mensaje, Toast.LENGTH_LONG).show();
```

Por defecto la ventana con el mensaje aparece posicionada en la parte de abajo y centrada horizontalmente, pero podemos posicionarla en cualquier parte del área de visualización usando del método **setGravity()**. Dicho método requiere como parámetros las coordenadas de la esquina superior izquierda de la ventana con el mensaje (objeto Toast).

Si queremos situarla en la esquina superior izquierda de la pantalla, pondremos los valores a 0:

```
toast.setGravity(Gravity.TOP|Gravity.LEFT, 0, 0);
```

Si queremos desplazar hacia abajo, cambiamos el primer 0 por un número mayor.

Si queremos desplazar hacia derecha, cambiamos el segundo 0 por un número mayor.

Para centrar horizontal y verticalmente en la pantalla:

```
toast.setGravity(Gravity.CENTER_VERTICAL, 0, 0);
```

En esta app mostramos un mensaje al usuario usando una ventana emergente mediante la clase Toast cuando el usuario clicca en el botón **Enviar mensaje**.

Apariencia:



[Google Developers](#) (Uso educativo nc)



[Google Developers](#) (Uso educativo nc)

Utilizaremos el método estático **makeText** de la clase **Toast** escribiendo toda la sentencia en una única línea.

strings.xml

```
1 <resources>
2     <string name="app_name">Proyecto 008. Ventana emergente. Clase Toast</string>
3     <string name="b_enviar_enviar">Enviar mensaje</string>
4     <string name="tv_mensaje_mensaje">¡Hola amig@s! Bienvenidos a este curso</str
5     <string name="resultado">Mensaje enviado</string>
6 </resources>
```

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <TextView
10         android:id="@+id/tv_mensaje"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:layout_centerHorizontal="true"
14         android:text="@string/tv_mensaje_mensaje"
15         android:textAlignment="center" />
16
17     <Button
18         android:id="@+id/b_enviar"
19         android:layout_width="wrap_content"
20         android:layout_height="wrap_content"
21         android:layout_below="@id/tv_mensaje"
22         android:layout_centerHorizontal="true"
23         android:onClick="enviarMensaje"
24         android:text="@string/b_enviar_enviar"
25         android:textAllCaps="false" />
26 </RelativeLayout>
27
```

MainActivity.java

```
1 package com.cidead.pmdm.proyecto008ventanaemergenteclasetoast;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.TextView;
8 import android.widget.Toast;
9
10 public class MainActivity extends AppCompatActivity {
11     @Override
12     protected void onCreate(Bundle savedInstanceState) {
13         super.onCreate(savedInstanceState);
14         setContentView(R.layout.activity_main);
15     }
16
17     public void enviarMensaje(View view) {
18         String mensaje=getResources().getString(R.string.resultado);
19         Toast.makeText(this, mensaje, Toast.LENGTH_LONG).show();
20     }
21 }
```

5.- Otras clases de formulario.

Caso práctico

Además de las clases Button, TextView y EditText, que son indispensables en el desarrollo de interfaces para aplicaciones de dispositivos móviles, hay otras clases que se utilizan con frecuencia y que podemos enmarcarlas dentro del conjunto de clases propias de los clásicos formularios de interacción con el usuario.

- Hasta ahora hemos trabajado con proyectos sencillos - explica Ada-. Voy a complicar las necesidades de estos pequeños proyectos.

- ¿Qué otras clases serán necesarias? -pregunta Pedro.

- Para las interfaces que os voy a pedir -indica Ada-, debéis utilizar RadioButtons, Checkboxes, Pickers y otros elementos típicos de formularios de interacción con los usuarios.

- ¡Claro! -exclama Pedro-. Son clases que se utilizan con mucha frecuencia.

- También es muy importante descubrir los distintos layouts con los que se pueden diseñar estas interfaces -remarca Ada-. Y, en el desarrollo de interfaces de usuarios, saber dar el foco al control adecuado en cada momento.



[Mikhail \(Pexels\)](#)

5.1.- Clases RadioGroup y RadioButton.

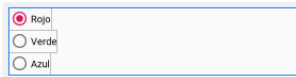
Los botones de radio ([RadioButton](#) ) permiten al usuario seleccionar una opción de un conjunto. Usaremos botones de opción (*radio buttons*) para conjuntos de opciones mutuamente excluyentes si nos parece oportuno que el usuario vea todas las opciones disponibles a la vez. Si no es necesario mostrar todas las opciones una al lado de la otra, es preferible utilizar un objeto de la clase [Spinner](#)  (lo veremos más adelante) en su lugar.

```
public class RadioGroup
    extends LinearLayout

    java.lang.Object
        ↳ android.view.View
            ↳ android.view.ViewGroup
                ↳ android.widget.LinearLayout
                    ↳ android.widget.RadioGroup
```

[Google Developers](#) (Uso educativo nc)

Para crear cada opción de botón de radio, debemos crear un objeto `RadioButton` en nuestro diseño. Sin embargo, como los botones de opción son mutuamente excluyentes, debemos agruparlos dentro de un grupo de botones de opciones ([RadioGroup](#) ). Al agruparlos, el sistema garantiza que solo se pueda seleccionar un botón de opción a la vez.



[Google Developers](#) (Uso educativo nc)

Nota: `RadioGroup` es una subclase de la clase `LinearLayout`, por lo que por defecto tendrá orientación vertical cómo puede verse en la imagen de la izquierda.

En este ejemplo, hemos marcamos como seleccionado el botón de opción Rojo usando:

```
android:checked="true"
```

Para marcar un botón de opción programáticamente podemos usar el método **`setChecked(boolean)`** sobre un objeto **`RadioButton`** deseado.

Podríamos también no marcar ningún botón de opción por defecto pero en nuestro caso vamos a dejar ese botón de opción marcado como opción por defecto.

strings.xml

```
1 <resources>
2   <string name="app_name">Proyecto 009. Eleccion de color. RadioGroup y RadioBu
3   <string name="rb_rojo">Rojo</string>
4   <string name="rb_verde">Verde</string>
5   <string name="rb_azul">Azul</string>
6 </resources>
```

main_activity.xml


```

1  <?xml version="1.0" encoding="utf-8"?>
2
3  <RadioGroup xmlns:android="http://schemas.android.com/apk/res/android"
4      android:id="@+id/rg_grupo1"
5      android:layout_width="fill_parent"
6      android:layout_height="wrap_content"
7      android:orientation="vertical">
8
9      <RadioButton
10         android:id="@+id/rb_rojo"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:checked="true"
14         android:text="@string/rb_rojo" />
15
16      <RadioButton
17         android:id="@+id/rb_verde"
18         android:layout_width="wrap_content"
19         android:layout_height="wrap_content"
20         android:text="@string/rb_verde" />
21
22      <RadioButton
23         android:id="@+id/rb_azul"
24         android:layout_width="wrap_content"
25         android:layout_height="wrap_content"
26         android:text="@string/rb_azul" />
27  </RadioGroup>
28

```

Cada botón de opción (objetos RadioButton) y el grupo de botones de opción (objeto RadioGroup) tienen un identificador asignado por nosotros que nos servirá en el código en Java para obtener una referencia a dichos objetos.

Ejemplo

```

1  RadioGroup rgGrupo1=(RadioGroup)findViewById(R.id.rg_grupo1);
2  RadioButton rbRojo=(RadioButton)findViewById(R.id.rb_rojo);
3  RadioButton rbVerde=(RadioButton)findViewById(R.id.rb_verde);
4  RadioButton rbAzul=(RadioButton)findViewById(R.id.rb_azul);

```

En la práctica, usaremos fundamentalmente la referencia al objeto RadioGroup para gestionar todo. Ya que a partir del grupo podremos obtener los botones de opción (objetos

RadioButton) como el identificador el botón de opción seleccionado en cada momento por el usuario.

Podríamos usar el método **getChildAt** de la clase **RadioGroup** para obtener cada botón contenido en un grupo de botones sin tener que especificar concretamente sus identificadores.

5.1.1.- Eventos en RadioGroup y RadioButton.

Cuando el usuario selecciona uno de los botones de opción (objetos **RadioButton**), el resto de botones de opción del mismo grupo de botones (objeto **RadioGroup**) quedan automáticamente deseleccionados.

Para gestionar eventos de botones de opción primero debemos saber que un botón de opción se comporta de una manera distinta a un botón normal (objeto **Button**) y por tanto **no funcionará** usar un objeto receptor del evento de clicado (**onClick**) que implemente la interfaz **OnClickListener** ya que un control **RadioButton** no está diseñado así.

Lo que tendremos que hacer es crear un objeto receptor de eventos que implemente la interfaz (estática) [OnCheckedChangeListener](#)  de la clase **RadioGroup**. Realizaremos la gestión del evento (responderemos al evento) implementando el método de retollamada:

```
public abstract void onCheckedChanged (RadioGroup group, int checkedId)
```

Este método es invocado cuando hay un cambio en la selección.

- ✓ **group**: objeto **RadioGroup** donde la selección ha cambiado.
- ✓ **checkedId**: identificador del botón de opción (objeto **RadioButton**) seleccionado o -1 si no hay ninguno seleccionado.

En el cuerpo del método de retollamada **onCheckedChanged** podemos obtener el botón de opción seleccionado usando:

```
group.findViewById(checkedId);
```

Nota: aparte, podremos preguntarle en cualquier momento al objeto **RadioGroup** cual es el botón de opción seleccionado usando su método: [getCheckedRadioButtonId](#)  .

5.1.2.- Ejercicio resuelto. Proyecto 9. Elección de color. RadioGroup y RadioButton.



[Google Developers](#) (Uso educativo nc)

Vamos a crear una app que muestre en una ventana emergente (control **Toast**) el nombre del color que ha seleccionado el usuario con color de texto del mismo color elegido.

La apariencia de la aplicación se muestra en la captura de la pantalla del móvil.

Código

```
1 <resources>
2   <string name="app_name">Proyecto 009. Eleccion de color. RadioGroup y RadioBu
3   <string name="rb_rojo">Rojo</string>
4   <string name="rb_verde">Verde</string>
5   <string name="rb_azul">Azul</string>
6 </resources>
```

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <RadioGroup xmlns:android="http://schemas.android.com/apk/res/android"
4   android:id="@+id/rg_grupo1"
5   android:layout_width="fill_parent"
6   android:layout_height="wrap_content"
7   android:orientation="vertical">
8
9   <RadioButton
10     android:id="@+id/rb_rojo"
11     android:layout_width="wrap_content"
```

```

12         android:layout_height="wrap_content"
13         android:checked="true"
14         android:text="@string/rb_rojo" />
15
16     <RadioButton
17         android:id="@+id/rb_verde"
18         android:layout_width="wrap_content"
19         android:layout_height="wrap_content"
20         android:text="@string/rb_verde" />
21
22     <RadioButton
23         android:id="@+id/rb_azul"
24         android:layout_width="wrap_content"
25         android:layout_height="wrap_content"
26         android:text="@string/rb_azul" />
27 </RadioGroup>

```

MainActivity.java

```

1 package com.cidead.pmdm.proyecto009elecciondecolorradiogrupyradiobutton;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.RadioButton;
8 import android.widget.RadioGroup;
9
10 public class MainActivity extends AppCompatActivity {
11     RadioGroup rgGrupo1;
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         setContentView(R.layout.activity_main);
17
18         rgGrupo1 =(RadioGroup)findViewById(R.id.rg_grupo1);
19         rgGrupo1.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener()
20             @Override
21             public void onCheckedChanged(RadioGroup group, int checkedId) {
22                 if (checkedId== -1)
23                     System.out.println("No hay seleccionado ningún color");
24                 else
25                     System.out.println("Color seleccionado: "
26                         +((RadioButton)group.findViewById(checkedId)).getText()
27                     );
28             });
29     }
30
31 }

```



Nota: en esta app no ha sido necesario pero podríamos usar el método [getId\(\)](#) heredado de la clase View para obtener el identificador de un objeto RadioGroup o RadioButton.

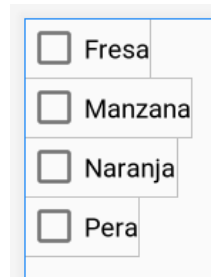
Recomendación

Se recomienda al alumnado como ejercicio terminar la app usando el objeto Toast.

5.2.- Clase CheckBox.

Las casillas de verificación (objetos [CheckBox](#) ) permiten al usuario seleccionar una o más opciones de un conjunto. Normalmente, se suelen mostrar las casillas de verificación en una lista vertical.

Para crear cada opción de casilla de verificación, debemos definir un elemento **CheckBox** en nuestro diseño. Debido a que un conjunto de opciones de casilla de verificación permite al usuario seleccionar múltiples elementos, cada casilla de verificación se administra por separado y debe registrar un receptor de eventos de para cada uno.



[Google Developers](#) (Uso educativo nc)

Hay que tener en cuenta en la imagen, que todos los elementos son seleccionables, siendo uno completamente independiente del otro.

5.2.1.- Eventos. Checkbox.

Cuando el usuario selecciona una casilla de verificación, se produce un evento de cambio de selección provocado por la interacción del usuario con el control (*widget*) `CheckBox`.

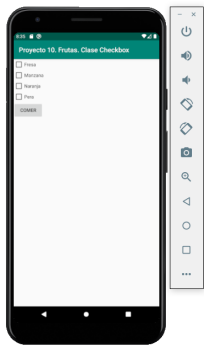
En este momento del curso debemos ser conscientes de que no debemos usar el evento `onClick` para objetos `RadioButton` ni `CheckBox` ya que Android ya tiene eventos e interfaces específicas que nos informan del cambio en las selecciones de opciones y esto es por una razón importante: **no nos importa la forma en la cual el usuario ha interactuado con esos botones para seleccionarlos sino el hecho de que la selección ha cambiado (podría haber ocurrido usando el teclado o el ratón, eso es irrelevante).**

Definiremos el método de retrollamada [`onCheckedChanged`](#)  en un receptor de eventos que será un subtipo de [`CompoundButton.OnCheckedChangeListener`](#) . Decimos bien que es un subtipo, ya que el receptor de eventos tendrá que implementar e implementará la interface **`OnCheckedChangeListener`**.

`CompoundButton` es una superclase de: [`CheckBox`](#) , [`RadioButton`](#) , [`Switch`](#)  y [`ToggleButton`](#) 

Vemos el ejemplo en el siguiente apartado donde se usa un receptor de eventos de clase anónima.

5.2.2.- Ejercicio resuelto. Proyecto 10. Frutas. Clase Checkbox.



Vamos a crear una app que muestra en un control de texto TextView las frutas seleccionadas como una lista de elementos separados por coma cuando se pulsa el botón **Comer**.

La apariencia de la aplicación se muestra en la captura de la pantalla del móvil.

[Google Developers](#) (Uso educativo
nc)

strings.xml

```
1 <resources>
2     <string name="app_name">Proyecto 10. Frutas. Clase Checkbox</string>
3     <string name="cb_fresa">Fresa</string>
4     <string name="cb_manzana">Manzana</string>
5     <string name="cb_naranja">Naranja</string>
6     <string name="cb_pera">Pera</string>
7     <string name="b_comer">Comer</string>
8 </resources>
```

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <CheckBox
10         android:id="@+id/cb_fresa"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
```

```

13         android:text="@string/cb_fresa"
14         tools:layout_editor_absoluteX="21dp"
15         tools:layout_editor_absoluteY="20dp" />
16
17     <CheckBox
18         android:id="@+id/cb_manzana"
19         android:layout_width="wrap_content"
20         android:layout_height="wrap_content"
21         android:layout_below="@id/cb_fresa"
22         android:text="@string/cb_manzana"
23         tools:layout_editor_absoluteX="21dp"
24         tools:layout_editor_absoluteY="20dp" />
25
26     <CheckBox
27         android:id="@+id/cb_naranja"
28         android:layout_width="wrap_content"
29         android:layout_height="wrap_content"
30         android:layout_below="@id/cb_manzana"
31         android:text="@string/cb_naranja"
32         tools:layout_editor_absoluteX="21dp"
33         tools:layout_editor_absoluteY="20dp" />
34
35     <CheckBox
36         android:id="@+id/cb_pera"
37         android:layout_width="wrap_content"
38         android:layout_height="wrap_content"
39         android:layout_below="@id/cb_naranja"
40         android:text="@string/cb_pera"
41         tools:layout_editor_absoluteX="21dp"
42         tools:layout_editor_absoluteY="20dp" />
43
44     <Button
45         android:id="@+id/b_comer"
46         android:layout_width="wrap_content"
47         android:layout_height="wrap_content"
48         android:layout_below="@id/cb_pera"
49         android:text="@string/b_comer" />
50 </RelativeLayout>

```

MainActivity.java

```

1 package com.cidead.pmdm.proyecto10frutascalsecheckbox;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.Button;
8 import android.widget.CheckBox;
9 import android.widget.CompoundButton;

```

```

10
11 import java.util.ArrayList;
12 import java.util.HashSet;
13
14 public class MainActivity extends AppCompatActivity {
15     Button bComer;
16     HashSet<String> frutasSel;
17
18     @Override
19     protected void onCreate(Bundle savedInstanceState) {
20         super.onCreate(savedInstanceState);
21         setContentView(R.layout.activity_main);
22
23         // Frutas seleccionadas
24         frutasSel=new HashSet<String>();
25
26         // Receptor de eventos para los objetos CheckBox
27         CompoundButton.OnCheckedChangeListener receptor=new CompoundButton.OnChec
28             @Override
29             public void onCheckedChanged(CompoundButton buttonView, boolean isChe
30                 if (isChecked)
31                     frutasSel.add(buttonView.getText().toString());
32                 else
33                     frutasSel.remove(buttonView.getText().toString());
34             }
35         };
36
37         // Registro del receptor en los objetos CheckBox
38         ArrayList<CheckBox> casillas=new ArrayList<>();
39         casillas.add((CheckBox)findViewById(R.id.cb_fresa));
40         casillas.add((CheckBox)findViewById(R.id.cb_manzana));
41         casillas.add((CheckBox)findViewById(R.id.cb_naranja));
42         casillas.add((CheckBox)findViewById(R.id.cb_pera));
43         for (CheckBox c: casillas)
44             c.setOnCheckedChangeListener(receptor);
45
46         // Botón Comer
47         bComer=(Button)findViewById(R.id.b_comer);
48         bComer.setOnClickListener(new View.OnClickListener() {
49             @Override
50             public void onClick(View v) {
51                 System.out.println(frutasSel.toString());
52             }
53         });
54     }
55 }

```

Recomendación

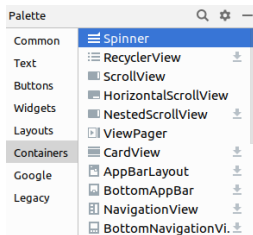
Se deja al alumnado terminar la app usando TextView.

5.3.- Clase Spinner.

El control **Spinner** ofrece una manera de seleccionar un valor de un conjunto en forma de lista desplegable. En el estado predeterminado, un control **Spinner** muestra su valor actualmente seleccionado. Al tocar el control **Spinner**, se muestra una lista los valores disponibles, de los cuales el usuario puede seleccionar uno.



[Google Developers](#) (Uso educativo nc)



En la paleta de controles de Android Studio aparece dentro de la categoría Containers (Contenedores), ya que es un control que puede contener otros elementos (como cadenas de texto por ejemplo).

Podemos agregar un control Spinner en nuestro diseño **XML** con un elemento **<Spinner>**.

[Google Developers](#) (Uso educativo nc)

Ejemplo

```
1 <Spinner
2     android:id="@+id/sp_semana"
3     android:layout_width="match_parent"
4     android:layout_height="wrap_content"
5     android:entries="@array/semana" />
```

El atributo **android:entries** sirve para especificar los elementos de la lista. En este caso lo hacemos mediante un array de cadenas (string-array) almacenada en **strings.xml**

strings.xml

```
1 <resources>
2     <string name="app_name">Proyecto 011. Días de la semana. Clase Spinner</string>
3     <string-array name="semana">
4         <item>Lunes</item>
5         <item>Martes</item>
6         <item>Miercoles</item>
7         <item>Jueves</item>
8         <item>Viernes</item>
9         <item>Sabado</item>
10        <item>Domingo</item>
11    </string-array>
```



5.3.1.- Carga de elementos seleccionables.

Los elementos de la lista podemos definirlos en **strings.xml** y asociarlos mediante el atributo **android:entries**. Por tanto, si las opciones disponibles para nuestro objeto Spinner son predeterminadas, podemos proporcionarlas con una array de strings definida en el archivo de recursos de strings **strings.xml** como vimos en el apartado anterior.

También es posible cargar los elementos de manera programática lo cual nos ofrece una mayor flexibilidad al poder hacerlo dinámicamente a partir de la fuente de datos que deseemos.

Las opciones que proporcionaremos para el control **Spinner** pueden provenir de cualquier fuente, pero deben brindarse mediante un objeto **SpinnerAdapter** (como por ejemplo un objeto **ArrayAdapter**) si están disponibles en un array o un **CursorAdapter** si están disponibles a partir de una consulta de la base de datos.

string-array

```
1 <resources>
2   <string name="app_name">Proyecto 011. Días de la semana. Clase Spinner</string>
3   <string-array name="semana">
4       <item>Lunes</item>
5       <item>Martes</item>
6       <item>Miercoles</item>
7       <item>Jueves</item>
8       <item>Viernes</item>
9       <item>Sabado</item>
10      <item>Domingo</item>
11   </string-array>
12 </resources>
```

Con un array como este, podemos usar el siguiente código en nuestra actividad para suministrar al control Spinner dicho array mediante una instancia de **ArrayAdapter**:

ArrayAdapter

```
1 Spinner spSemana = (Spinner)findViewById(R.id.sp_semana);
2
3 // Creamos un objeto ArrayAdapter usando el array de strings y un gestor de coloc
4
```

```
5 | ArrayAdapter<CharSequence> adaptador = ArrayAdapter.createFromResource(this, R.ar
6 |
7 | // Especificamos el gestor de colocación (layout) que usaremos cuando aparezcan l
8 | adaptador.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
9 |
10 | // Aplicamos el adaptador al Spinner
11 | spSemana.setAdapter(adaptador);
```

El método **createFromResource()** permite crear un objeto **ArrayAdapter** a partir del array de strings. El tercer argumento para este método es un recurso de diseño que define la manera en que se muestra la opción seleccionada en el control **Spinner**. El diseño de **simple_spinner_item** se proporciona mediante la plataforma y es el diseño predeterminado que debes usar a menos que desees definir tu propio diseño para la apariencia del control **Spinner**.

Luego, hay que invocar **setDropDownViewResource(int)** a fin de especificar el diseño que debe usar el adaptador para mostrar la lista de elecciones del control de número (**simple_spinner_dropdown_item** es otro diseño estándar definido por la plataforma).

Llamaremos finalmente a **setAdapter()** para aplicar el adaptador a nuestro control **Spinner**.

5.3.2.- Gestión de eventos de selección.

Cuando un usuario selecciona un elemento de menú desplegable, el objeto **Spinner** recibe un evento en relación con el elemento seleccionado.

Para definir el controlador del evento de selección para un **Spinner**, implementaremos la interfaz **AdapterView.OnItemSelectedListener** y el método de retrollamada **onItemSelected()** correspondiente.

Ejemplo

Implementación de la interfaz en una actividad

```
1 public class MainActivity implements OnItemSelectedListener {
2     ...
3     public void onItemSelected(AdapterView<?> parent, View view,
4         int pos, long id) {
5         // Un elemento fue seleccionado. Podemos obtener el elemento seleccionado
6     }
7     public void onNothingSelected(AdapterView<?> parent) {
8         // Ningun elemento seleccionado
9     }
10 }
```

AdapterView.OnItemSelectedListener tiene los métodos de retrollamada **onItemSelected()** y **onNothingSelected()**.

Debemos especificar la implementación de la interfaz; para ello, llamaremos a **setOnItemSelectedListener()**.

setOnItemSelectedListener()

```
1 Spinner spSemana = (Spinner)findViewById(R.id.sp_semana);
2
3 spSemana.setOnItemSelectedListener(this);
```

5.3.3.- Ejercicio resuelto. Proyecto 11. Días de la semana. Clase Spinner.



Vamos a crear una app que permita seleccionar un día de la semana de una lista desplegable usando un control Spinner.

Si el día de la semana es Viernes entonces se mostrará en una ventana emergente: "¡Al fin viernes! :)"

La apariencia de la aplicación se muestra en captura de la pantalla del móvil.

[Google Developers](#) (Uso educativo nc)

strings.xml

```
1  <resources>
2      <string name="app_name">Proyecto 011. Dias de la semana. Clase Spinner</string>
3      <string-array name="semana">
4          <item>Lunes</item>
5          <item>Martes</item>
6          <item>Miercoles</item>
7          <item>Jueves</item>
8          <item>Viernes</item>
9          <item>Sabado</item>
10         <item>Domingo</item>
11     </string-array>
12 </resources>
```

activity_main.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      tools:context=".MainActivity">
7
```

```

8         <Spinner
9             android:id="@+id/sp_semana"
10            android:layout_width="match_parent"
11            android:layout_height="wrap_content" />
12    </RelativeLayout>

```

MainActivity.java

```

1  package com.cidead.pmdm.proyecto011diasdelasemanaclasespinner;
2
3  import androidx.appcompat.app.AppCompatActivity;
4
5  import android.os.Bundle;
6  import android.view.View;
7  import android.widget.AdapterView;
8  import android.widget.ArrayAdapter;
9  import android.widget.Spinner;
10 import android.widget.Toast;
11
12 public class MainActivity extends AppCompatActivity
13     implements AdapterView.OnItemClickListener {
14
15     Spinner spSemana;
16
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         setContentView(R.layout.activity_main);
21
22         // Control Spinner
23         spSemana=(Spinner)findViewById(R.id.sp_semana);
24
25         // Creamos un objeto ArrayAdapter usando el array de strings
26         // y un gestor de colocación (layout) por defecto para el control Spinner
27         ArrayAdapter<CharSequence> adaptador = ArrayAdapter.createFromResource(this,
28             R.array.semana, android.R.layout.simple_spinner_item);
29
30         // Especificamos el gestor de colocación (layout) que usaremos
31         // cuando aparezcan la lista de elementos
32         adaptador.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
33
34         // Aplicamos el adaptador al Spinner
35         spSemana.setAdapter(adaptador);
36
37         // Registramos la propia clase MainActivity como receptor de eventos en el spinner
38         spSemana.setOnItemClickListener(this);
39     }
40
41     @Override
42     public void onItemClick(AdapterView<?> parent, View view, int position, long id) {

```

```
43         Toast.makeText(parent.getContext(), parent.getItemAtPosition(position).tc
44             Toast.LENGTH_SHORT).show();
45     }
46
47     @Override
48     public void onNothingSelected(AdapterView<?> parent) {
49         Toast.makeText(parent.getContext(),
50             "Ningún elemento seleccionado",
51             Toast.LENGTH_SHORT).show();
52     }
53 }
```

Recomendación

Se deja al alumnado completar el ejercicio para que en el caso del viernes se muestre el mensaje indicado en el enunciado y en el resto de casos se muestre simplemente el nombre del día de la semana seleccionado por el usuario.

5.4.- Clases `ToggleButton` y `Switch`.

La clase [`ToggleButton`](#)  corresponde a un tipo de botón que permite al usuario alternar entre dos estados.

Este botón de transición entre 2 estados es relativamente primitivo o básico. Android 4.0 (nivel de API 14) introduce otro tipo de botón de alternancia de estados llamado [`Switch`](#)  que proporciona un control deslizante. [`SwitchCompat`](#)  es una versión del control `Switch` que se ejecuta en dispositivos que dispongan de API 7 (o mayor).

Si necesitamos cambiar el estado de un botón, podemos usar los métodos [`setChecked\(\)`](#)  o [`toggle\(\)`](#)  de [`CompoundButton`](#) .

5.4.1.- Eventos.

Para detectar cuándo el usuario activa o desactiva el botón (objeto **Toggle**) o el interruptor (objeto **Switch**), crearemos un receptor de eventos implementando **CompoundButton.OnCheckedChangeListener** y lo registramos en el botón con **setOnCheckedChangeListener()**.

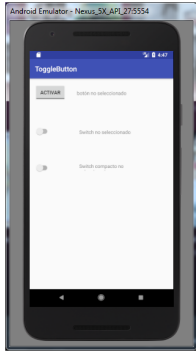
Ejemplo

```
1 ToggleButton toggle = (ToggleButton) findViewById(R.id.togglebutton);
2 toggle.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener() {
3     public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
4         if (isChecked) {
5             // The toggle is enabled
6         } else {
7             // The toggle is disabled
8         }
9     }
10 });
```

Para un control Switch lo haremos de forma equivalente.

5.4.2.- Ejercicio resuelto. Proyecto 12.

Botones de alternancia. Clases ToggleButton, Switch y SwitchCompat.



Crear una pequeña aplicación donde se incorporen todas las variantes de elementos de alternancia, esto es, ToggleButton, Switch y SwitchCompat.

La apariencia de la aplicación se muestra en la captura de la pantalla del móvil.

Crear un nuevo proyecto en Android Studio: Proyecto 12. Botones de alternancia. Clases ToggleButton, Switch y SwitchCompat

[Google Developers](#) (Uso educativo nc)

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     tools:context=".MainActivity">
9
10    <ToggleButton
11        android:id="@+id/tb"
12        android:layout_width="wrap_content"
13        android:layout_height="wrap_content"
14        android:layout_marginLeft="16dp"
15        android:layout_marginTop="16dp"
16        android:checked="false"
17        android:textOff="activar"
18        android:textOn="desactivar"
19        app:layout_constraintLeft_toLeftOf="parent"
20        app:layout_constraintTop_toTopOf="parent"
21        tools:ignore="MissingConstraints" />
22
23    <TextView
24        android:id="@+id/tv1"
25        android:layout_width="wrap_content"
26        android:layout_height="wrap_content"
27        android:layout_marginStart="32dp"
28        android:layout_marginTop="32dp"
29        android:text="botón no seleccionado"
```

```

30         app:layout_constraintStart_toEndOf="@+id/tb"
31         app:layout_constraintTop_toTopOf="parent"
32         android:layout_marginLeft="32dp" />
33
34     <Switch
35         android:id="@+id/sw"
36         android:layout_width="wrap_content"
37         android:layout_height="23dp"
38         android:layout_marginStart="16dp"
39         android:layout_marginTop="74dp"
40         app:layout_constraintStart_toStartOf="parent"
41         app:layout_constraintTop_toBottomOf="@+id/tb"
42         tools:ignore="MissingConstraints"
43         android:layout_marginLeft="16dp" />
44
45     <TextView
46         android:id="@+id/tv2"
47         android:layout_width="150dp"
48         android:layout_height="20dp"
49         android:layout_marginStart="80dp"
50         android:layout_marginTop="92dp"
51         android:text="Switch no seleccionado"
52         app:layout_constraintStart_toEndOf="@+id/sw"
53         app:layout_constraintTop_toBottomOf="@+id/tv1"
54         tools:ignore="MissingConstraints"
55         android:layout_marginLeft="80dp" />
56
57     <android.support.v7.widget.SwitchCompat
58         android:id="@+id/swc"
59         android:layout_width="wrap_content"
60         android:layout_height="wrap_content"
61         android:layout_marginLeft="16dp"
62         android:layout_marginStart="16dp"
63         app:layout_constraintBottom_toBottomOf="parent"
64         app:layout_constraintLeft_toLeftOf="parent"
65         app:layout_constraintStart_toStartOf="parent"
66         app:layout_constraintTop_toBottomOf="@+id/sw"
67         app:layout_constraintVertical_bias="0.188"
68         tools:ignore="MissingConstraints" />
69
70     <TextView
71         android:id="@+id/tv3"
72         android:layout_width="212dp"
73         android:layout_height="23dp"
74         android:layout_marginStart="80dp"
75         android:layout_marginTop="8dp"
76         android:text="Switch compacto no seleccionado"
77         app:layout_constraintBottom_toBottomOf="parent"
78         app:layout_constraintStart_toEndOf="@+id/swc"
79         app:layout_constraintTop_toBottomOf="@+id/tv2"
80         app:layout_constraintVertical_bias="0.17"
81         android:layout_marginLeft="80dp" />
82 </android.support.constraint.ConstraintLayout>

```


MainActivity.java

```
1  package com.cidead.pmdm.proyecto012botonesdealternanciaclasesetogglebuttonswitchys
2
3  import android.graphics.Color;
4  import android.support.v7.app.AppCompatActivity;
5  import android.os.Bundle;
6  import android.support.v7.widget.SwitchCompat;
7  import android.widget.CompoundButton;
8  import android.widget.Switch;
9  import android.widget.TextView;
10 import android.widget.ToggleButton;
11
12 public class MainActivity extends AppCompatActivity {
13     ToggleButton tb;
14     Switch sw;
15     SwitchCompat swc;
16     TextView tv1, tv2, tv3;
17
18     @Override
19     protected void onCreate(Bundle savedInstanceState) {
20         super.onCreate(savedInstanceState);
21         setContentView(R.layout.activity_main);
22         tb=(ToggleButton)findViewById(R.id.tb);
23         sw=(Switch)findViewById(R.id.sw);
24         swc=(SwitchCompat)findViewById(R.id.swc);
25         tv1=(TextView)findViewById(R.id.tv1);
26         tv2=(TextView)findViewById(R.id.tv2);
27         tv3=(TextView)findViewById(R.id.tv3);
28
29         tb.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener(
30             @Override
31             public void onCheckedChanged(CompoundButton compoundButton, boolean b) {
32                 if (b) { //Si está marcado
33                     tv1.setText("botón seleccionado");
34                     tv1.setTextColor(Color.RED);
35                 } else {
36                     tv1.setText("botón no seleccionado");
37                     tv1.setTextColor(Color.GRAY);
38                 }
39             }
40         ));
41
42         sw.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener(
43             @Override
44             public void onCheckedChanged(CompoundButton compoundButton, boolean b) {
45                 if (b) { //Si está marcado
46                     tv2.setText("Switch seleccionado");
47                     tv2.setTextColor(Color.RED);
48                 } else {
49                     tv2.setText("Switch no seleccionado");
50                     tv2.setTextColor(Color.GRAY);
51                 }
52             }
53         ));
54     }
55 }
```

```

52         }
53     });
54
55     swc.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener
56         @Override
57         public void onCheckedChanged(CompoundButton compoundButton, boolean b) {
58         if (b) { //Si está marcado
59             tv3.setText("Switch compacto seleccionado");
60             tv3.setTextColor(Color.RED);
61         } else {
62             tv3.setText("Switch compacto no seleccionado");
63             tv3.setTextColor(Color.GRAY);
64         }
65     }
66 });
67 }
68 }

```

Recomendación

Se recomienda al alumnado la creación y uso de cadenas en **strings.xml** en vez del uso de literales de cadena en **activity_main.xml** y **MainActivity.java** .

5.5.- Clases selectoras de fechas y horas.

Android proporciona controles para que el usuario elija una hora o elija una fecha como diálogos listos para usar. Cada selector proporciona controles para seleccionar de manera independiente cada parte del tiempo (hora, minuto, am/pm) o fecha (mes, día, año). El uso de estos selectores ayuda a garantizar que los usuarios puedan elegir una hora o fecha válida, formateada correctamente y ajustada a la configuración regional del usuario.

Se recomienda utilizar [DialogFragment](#)  para alojar cada selector de fecha y hora. **DialogFragment** gestiona el ciclo de vida del diálogo y permite mostrar los selectores en diferentes configuraciones de diseño, como en un cuadro de diálogo básico o como una parte integrada del diseño en pantallas grandes.

5.5.1.- Creación de selector de tiempo.

Para mostrar un control **TimePickerDialog** utilizando **DialogFragment**, necesitamos definir una clase de fragmento que extienda **DialogFragment** y devolver un objeto **TimePickerDialog** desde el método **onCreateDialog ()** del fragmento.

1. Cómo extender DialogFragment para un selector de tiempo

Para definir un fragmento con ventana de diálogo (**DialogFragment**) para un control **TimePickerDialog**, debemos:

- ✓ Definir el método **onCreateDialog ()** para devolver una instancia de **TimePickerDialog**
- ✓ Implementar la interfaz **TimePickerDialog.OnTimeSetListener** para recibir una devolución de llamada cuando el usuario establece la hora.

Ejemplo

```
1 public static class TimePickerFragment extends DialogFragment
2 implements TimePickerDialog.OnTimeSetListener {
3     @Override
4     public Dialog onCreateDialog(Bundle savedInstanceState) {
5         // Use the current time as the default values for the picker
6         final Calendar c = Calendar.getInstance();
7         int hour = c.get(Calendar.HOUR_OF_DAY);
8         int minute = c.get(Calendar.MINUTE);
9
10        // Create a new instance of TimePickerDialog and return it
11        return new TimePickerDialog(getActivity(), this, hour, minute,
12            DateFormat.is24HourFormat(getActivity()));
13    }
14
15    public void onTimeSet(TimePicker view, int hourOfDay, int minute) {
16        // Do something with the time chosen by the user
17    }
18 }
```

Ahora todo lo que necesitamos es un evento que agregue una instancia de este fragmento a nuestra actividad.

2. Cómo mostrar el selector de tiempo

Una vez que hayamos definido un fragmento con ventana de diálogo (**DialogFragment**) como el que se muestra arriba, podemos mostrar el selector de tiempo creando una instancia de **DialogFragment** y llamar al método **show()**.

Ejemplo

Aquí hay un botón que, cuando se hace clic, llama a un método para mostrar el diálogo

```
1 <Button
2     android: layout_width = "wrap_content"
3     android: layout_height = "wrap_content"
4     android: text = "@string/pick_time"
5     android: onClick = "showTimePickerDialog" />
6
7 Cuando el usuario hace clic en este botón, el sistema llama al siguiente método:
8
9 public void showTimePickerDialog (View v) {
10     DialogFragment newFragment = new TimePickerFragment ();
11     newFragment.show (getSupportFragmentManager (), "timePicker");
12 }
13
```

Este método llama a **show ()** en una nueva instancia del **DialogFragment** definido anteriormente. El método **show()** requiere una instancia de **FragmentManager** y un nombre de etiqueta único para el fragmento.

Precaución: si su aplicación es compatible con versiones de Android inferiores a 3.0, debemos asegurarnos de llamar a **getSupportFragmentManager ()** para adquirir una instancia de **FragmentManager**. También hay que asegurarse de que nuestra actividad que muestra el selector de tiempo extienda **FragmentActivity** en lugar de la clase de actividad estándar.

5.5.2.- Creación de selector de fecha.

Crear un diálogo **DatePickerDialog** es como crear un diálogo **TimePickerDialog**. La única diferencia es el diálogo que creas para el fragmento. Para mostrar un diálogo **DatePickerDialog** usando **DialogFragment**, necesitas definir una clase de fragmento que extienda **DialogFragment** y devolver un objeto **DatePickerDialog** desde el método **onCreateDialog()** del fragmento.

1. Cómo extender DialogFragment para un selector de fecha:

Para definir un fragmento **DialogFragment** para un diálogo **DatePickerDialog**, debemos:

- ✓ Definir el método **onCreateDialog()** para devolver una instancia de **DatePickerDialog**
- ✓ Implementar la interfaz **DatePickerDialog.OnDateSetListener** para recibir una devolución de llamada cuando el usuario establece la fecha.

Ejemplo

```
1 public static class DatePickerFragment extends DialogFragment
2 implements DatePickerDialog.OnDateSetListener
3 {
4     @Override
5     public Dialog onCreateDialog(Bundle savedInstanceState) {
6         // Use the current date as the default date in the picker
7         final Calendar c = Calendar.getInstance();
8         int year = c.get(Calendar.YEAR);
9         int month = c.get(Calendar.MONTH);
10        int day = c.get(Calendar.DAY_OF_MONTH);
11
12        // Create a new instance of DatePickerDialog and return it
13        return new DatePickerDialog(getActivity(), this, year, month, day);
14    }
15
16    public void onDateSet(DatePicker view, int year, int month, int day) {
17        // Do something with the date chosen by the user
18    }
19 }
```

Ahora todo lo que necesitamos es un evento que agregue una instancia de este fragmento a nuestra actividad.

2. Cómo mostrar el selector de fecha.

Una vez que hayamos definido un fragmento **DialogFragment** como el que se muestra arriba, podemos mostrar el selector de tiempo creando una instancia de **DialogFragment** y llamar al método **show()**.

Ejemplo

Aquí hay un botón que, cuando se hace clic, llama a un método para mostrar el diálogo.

```
1 <Button
2     android:layout_width="wrap_content"
3     android:layout_height="wrap_content"
4     android:text="@string/pick_date"
5     android:onClick="showDatePickerDialog" />
```

Cuando el usuario hace clic en este botón, el sistema llama al siguiente método:

```
1 public void showDatePickerDialog(View v) {
2     DialogFragment newFragment = new DatePickerFragment();
3     newFragment.show(getSupportFragmentManager(), "datePicker");
}
```

Este método llama a **show()** en una nueva instancia del **DialogFragment** definido anteriormente. El método **show()** requiere una instancia de **FragmentManager** y un nombre de etiqueta único para el fragmento.

5.5.3.- Ejercicio resuelto. Proyecto 13. Selector de tiempo. Clase TimePicker.

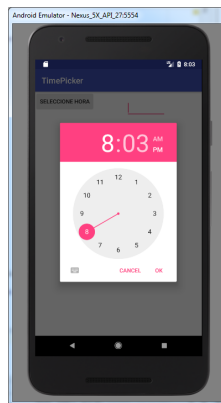
Crear un botón que lance un diálogo TimePickerDialog y que manejará un TimePicker para poder seleccionar una hora. Por sencillez no usaremos fragmentos (concepto que veremos más adelante en el curso):

Apariencia inicial:



[Google Developers](#) (Uso educativo nc)

Al clicar en el botón:



[Google Developers](#) (Uso educativo nc)

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
```



```

5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <Button
10         android:id="@+id/btnHora"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="Seleccione hora" />
14
15     <EditText
16         android:id="@+id/etHora"
17         android:layout_width="97dp"
18         android:layout_height="wrap_content"
19         app:layout_constraintBottom_toBottomOf="parent"
20         app:layout_constraintHorizontal_bias="0.717"
21         app:layout_constraintLeft_toLeftOf="parent"
22         app:layout_constraintRight_toRightOf="parent"
23         app:layout_constraintTop_toTopOf="parent"
24         app:layout_constraintVertical_bias="0.032" />
25 </android.support.constraint.ConstraintLayout>

```

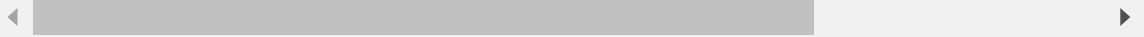
MainActivity.java

```

1 package com.cidead.pmdm.proyecto013selectordetiempoclasetimepicker
2
3 import android.app.TimePickerDialog;
4 import android.support.v7.app.AppCompatActivity;
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.Button;
8 import android.widget.EditText;
9 import android.widget.TimePicker;
10 import java.util.Calendar;
11
12 public class MainActivity extends AppCompatActivity implements View.OnClickListener
13     private Button btnHora;
14     private EditText etHora;
15     private int hora, minutos;
16
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         setContentView(R.layout.activity_main);
21
22         btnHora=(Button)findViewById(R.id.btnHora);
23         etHora=(EditText)findViewById(R.id.etHora);
24         btnHora.setOnClickListener(this);
25     }

```

```
26
27     @Override
28     public void onClick(View view) {
29         final Calendar c= Calendar.getInstance();
30         hora=c.get(Calendar.HOUR_OF_DAY);
31         minutos=c.get(Calendar.MINUTE);
32         TimePickerDialog timePickerDialog=new TimePickerDialog(this, new TimePickerDialog
33             @Override
34             public void onTimeSet(TimePicker timePicker, int hour, int minute)
35                 etHora.setText(hour+":"+minute);
36             },
37             hora, minutos, false);
38
39         // Para que inicialmente cargue en el reloj la hora del sistema
40         // (aunque esto lo hace por defecto)
41         timePickerDialog.updateTime(hora, minutos);
42
43         timePickerDialog.show();
44     }
45 }
```



5.5.4.- Ejercicio resuelto. Proyecto 14. Selector de fecha. Clase DatePicker.

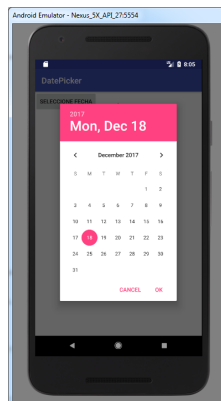
Crear un botón que lance un diálogo DatePickerDialog y que manejará un control DatePicker para poder seleccionar una hora. Por sencillez tampoco usaremos Fragments:

Apariencia inicial:



[Google Developers](#) (Uso educativo nc)

Al clicar en el botón:



[Google Developers](#) (Uso educativo nc)

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
```

```

6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      tools:context=".MainActivity">
9
10     <Button
11         android:id="@+id/btnFecha"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="Seleccione fecha" />
15
16     <EditText
17         android:id="@+id/etFecha"
18         android:layout_width="132dp"
19         android:layout_height="wrap_content"
20         app:layout_constraintBottom_toBottomOf="parent"
21         app:layout_constraintHorizontal_bias="0.717"
22         app:layout_constraintLeft_toLeftOf="parent"
23         app:layout_constraintRight_toRightOf="parent"
24         app:layout_constraintTop_toTopOf="parent"
25         app:layout_constraintVertical_bias="0.032" />
26 </android.support.constraint.ConstraintLayout>

```

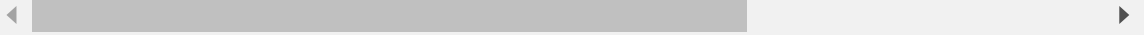
MainActivity.java

```

1  package com.cidead.pmdm.proyecto14selectordefechaclasedatepicker;
2
3  import android.app.DatePickerDialog;
4  import android.support.v7.app.AppCompatActivity;
5  import android.os.Bundle;
6  import android.view.View;
7  import android.widget.Button;
8  import android.widget.DatePicker;
9  import android.widget.EditText;
10 import java.util.Calendar;
11
12 public class MainActivity extends AppCompatActivity implements View.OnClickListener
13     private Button btnFecha;
14     private EditText etFecha;
15     private int dia, mes, anno;
16
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         setContentView(R.layout.activity_main);
21
22         btnFecha=(Button)findViewById(R.id.btnFecha);
23         etFecha=(EditText)findViewById(R.id.etFecha);
24         btnFecha.setOnClickListener(this);
25     }

```

```
26
27     @Override
28     public void onClick(View view) {
29         final Calendar c= Calendar.getInstance();
30         dia=c.get(Calendar.DAY_OF_MONTH);
31         mes=c.get(Calendar.MONTH);
32         anno=c.get(Calendar.YEAR);
33
34         DatePickerDialog datePickerDialog=new DatePickerDialog(this, new DatePickerDialog.OnDateSetListener() {
35             @Override
36             public void onDateSet(DatePicker datePicker, int year, int month, int day) {
37                 etFecha.setText(day+"/"+(month+1)+"/"+year);
38             }
39         }, dia, mes, anno);
40
41         datePickerDialog.updateDate(anno, mes, dia); //Para que inicialmente cargue la fecha actual
42         datePickerDialog.show();
43     }
44 }
```



5.6.- Clase ImageButton.

La clase ImageButton mostrará un botón con una imagen (en lugar de texto) que el usuario puede presionar o hacer clic. De forma predeterminada, un ImageButton se parece a un botón normal, con el fondo de botón estándar que cambia de color durante diferentes estados de botón. La imagen en la superficie del botón está definida por el atributo **android:src** en el elemento **<ImageButton>** XML o por el método **setImageResource (int)**.



[Google Developers](#) (Uso educativo no)

Para eliminar la imagen de fondo del botón estándar, definiremos nuestra propia imagen de fondo o configuraremos el color de fondo para que sea transparente.

Para indicar los diferentes estados de los botones (enfocado, seleccionado, etc.), podemos definir una imagen diferente para cada estado. Por ejemplo, una imagen azul por defecto, una naranja para cuando se enfoca, y una amarilla para cuando se presiona. Una forma fácil de hacerlo es con un "selector" XML dibujable.

Ejemplo

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <selector xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:state_pressed="true"
4         android:drawable="@drawable/button_pressed" /> <!-- pressed -->
5     <item android:state_focused="true"
6         android:drawable="@drawable/button_focused" /> <!-- focused -->
7     <item android:drawable="@drawable/button_normal" /> <!-- default -->
8 </selector>
```

Guardaremos el archivo XML en la carpeta **res/drawable/** del proyecto y luego haremos referencia como un dibujable (*drawable*) para el origen (en el atributo **android:src**). Android cambiará automáticamente la imagen según el estado del botón y las imágenes correspondientes definidas en el fichero XML.

El orden de los elementos **<item>** es importante porque se evalúan en orden. Esta es la razón por la cual la imagen del botón "normal" viene en último lugar, porque solo se aplicará después de que **android:state_pressed** y **android: state_focused** hayan sido ambos evaluados como falsos.

5.6.1.- Ejercicio resuelto. Proyecto 15. Botón con imagen. Clase ImageButton.

Siguiendo la pauta indicada en el apartado anterior, crear una aplicación en la que aparezca la imagen de un teléfono, y esta cambie de color dependiendo de si ha sido clicada, si tiene el foco de la aplicación (se obtiene con el tabulador) o si no se ha hecho nada sobre ella.

Estado de reposo.



[Google Developers](#) (Uso educativo nc)

Estado con el foco.



[Google Developers](#) (Uso educativo nc)

Estado clicado.



[Google Developers](#) (Uso educativo nc)

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     android:orientation="vertical"
9     tools:context=".MainActivity">
10
11     <TextView
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="Estados de ImageButton"
15         android:layout_gravity="center"/>
16
17     <ImageButton
18         android:layout_width="wrap_content"
19         android:layout_height="wrap_content"
20         android:layout_gravity="center"
21         android:layout_marginTop="50dp"
22         android:src="@drawable/estados"/> <!-- En lugar de seleccionar una imager
23 </LinearLayout>
```

Creamos un fichero **estados.xml** en app | res | drawable clicando con el botón derecho del ratón sobre la opción "New | Drawable Resource File" sobre la carpeta "**drawable**":

estados.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <selector xmlns:android="http://schemas.android.com/apk/res/android">
4     <item android:state_pressed="true"
5         android:drawable="@drawable/telefono_rojo" /> <!-- clicado -->
6     <item android:state_focused="true"
7         android:drawable="@drawable/telefono_naranja" /> <!-- cuando con el tabul
8     <item android:drawable="@drawable/telefono_verde" /> <!-- por defecto, sin tc
9 </selector>
```

MainActivity.java

```
1 package com.cidead.pmdm.proyecto015botonconimagenclaseimagebutton;
2
3 import android.support.v7.app.AppCompatActivity;
4 import android.os.Bundle;
5
6 public class MainActivity extends AppCompatActivity {
7     @Override
8     protected void onCreate(Bundle savedInstanceState) {
9         super.onCreate(savedInstanceState);
10        setContentView(R.layout.activity_main);
11    }
12 }
```

6.- Foco.

Caso práctico



[Mikhail](#) ([Pexels](#))

- Ya hemos revisado las clases que se utilizan con frecuencia en el desarrollo de interfaces para dispositivos móviles -explica Ada-. Ahora vamos a ver cómo manejar el foco.

- En el desarrollo de interfaces de usuarios -expone María-, debemos dar el foco al control adecuado en cada momento.

- Exacto -afirma Ada-. El foco es muy importante en la interacción de la aplicación con los usuarios.

6.1.- Manejo del foco.

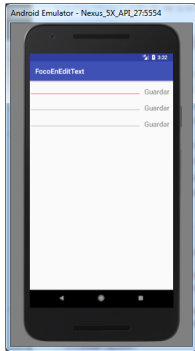
El framework manejará el movimiento del foco de rutina en respuesta a las entradas del usuario. Esto incluye cambiar el foco a medida que se eliminan o se ocultan las vistas, o a medida que existen vistas nuevas disponibles. Las vistas indican su disposición a tomar foco a través del método **isFocusable()**. Para cambiar si una vista puede tomar foco, llama a **setFocusable()**. En el modo táctil, puedes consultar si una vista permite el foco con **isFocusableInTouchMode()**. Puedes cambiar esto con **setFocusableInTouchMode()**.

Si deseas declarar una vista como enfocable en tu IU (cuando por defecto no lo es), agregaremos el atributo XML **android:focusable** a la vista en nuestra declaración de diseño. Fijaremos el valor en *true*. También puedes declarar una vista como enfocable en el modo táctil con **android:focusableInTouchMode**.

Para solicitar que se enfoque una vista en particular, llama a **requestFocus()**.

Para escuchar eventos de foco (ser notificado cuando una vista recibe o pierde foco), utilizaremos **onFocusChange()**.

6.2.- Ejercicio resuelto. Proyecto 16. Foco.



Crear una aplicación que gestione el manejo del foco.

La apariencia de la aplicación se muestra en la captura de la pantalla del móvil.

Al pulsar sobre los controles TextView, el foco saltará al siguiente control EditText. Además el foco pasará del último EditText al primero.

[Google Developers](#) (Uso educativo nc)

activity_main.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2
3  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      android:orientation="vertical"
9      tools:context=".MainActivity">
10
11      <LinearLayout
12          android:layout_width="match_parent"
13          android:layout_height="wrap_content"
14          android:orientation="horizontal">
15
16          <!-- Foco por defecto aquí -->
17          <EditText
18              android:layout_width="match_parent"
19              android:layout_height="wrap_content"
20              android:id="@+id/et1"
21              android:layout_weight="0.1"
22              android:focusableInTouchMode="true" />
23
24          <TextView
25              android:layout_width="wrap_content"
26              android:layout_height="wrap_content"
27              android:id="@+id/tv1"
28              android:text="Guardar"
29              android:textSize="20sp"
30              android:layout_margin="10dp"/>
31      </LinearLayout>
32
33      <LinearLayout
```

```

34         android:layout_width="match_parent"
35         android:layout_height="wrap_content"
36         android:orientation="horizontal">
37
38         <EditText
39             android:layout_width="match_parent"
40             android:layout_height="wrap_content"
41             android:id="@+id/et2"
42             android:layout_weight="0.1" />
43
44         <TextView
45             android:layout_width="wrap_content"
46             android:layout_height="wrap_content"
47             android:text="Guardar"
48             android:id="@+id/tv2"
49             android:textSize="20sp"
50             android:layout_margin="10dp"/>
51     </LinearLayout>
52
53     <LinearLayout
54         android:layout_width="match_parent"
55         android:layout_height="wrap_content"
56         android:orientation="horizontal">
57
58         <EditText
59             android:layout_width="match_parent"
60             android:layout_height="wrap_content"
61             android:id="@+id/et3"
62             android:layout_weight="0.1" />
63
64         <TextView
65             android:layout_width="wrap_content"
66             android:layout_height="wrap_content"
67             android:text="Guardar"
68             android:id="@+id/tv3"
69             android:textSize="20sp"
70             android:layout_margin="10dp"/>
71     </LinearLayout>
72 </LinearLayout>

```

MainActivity.java

```

1 package com.cidead.pmdm.proyecto016foco;
2
3 import android.support.v7.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.view.View;
6 import android.widget.EditText;
7 import android.widget.TextView;
8

```

```

9 public class MainActivity extends AppCompatActivity {
10     EditText et1, et2, et3;
11     TextView tv1, tv2, tv3;
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         setContentView(R.layout.activity_main);
17
18         et1 = (EditText) findViewById(R.id.et1);
19         et2 = (EditText) findViewById(R.id.et2);
20         et3 = (EditText) findViewById(R.id.et3);
21
22         tv1 = (TextView) findViewById(R.id.tv1);
23         tv2 = (TextView) findViewById(R.id.tv2);
24         tv3 = (TextView) findViewById(R.id.tv3);
25
26         tv1.setOnClickListener(new View.OnClickListener() {
27             @Override
28             public void onClick(View v) {
29                 et1.clearFocus();
30                 et2.requestFocus();
31
32                 String str = et1.getText().toString();
33                 Toast msg = Toast.makeText(getApplicationContext(), str, Toast.LENGTH_LONG);
34                 msg.show();
35             }
36         });
37
38         tv2.setOnClickListener(new View.OnClickListener() {
39             @Override
40             public void onClick(View v) {
41                 et2.clearFocus();
42                 et3.requestFocus();
43
44                 String str = et2.getText().toString();
45                 Toast msg = Toast.makeText(getApplicationContext(), str, Toast.LENGTH_LONG);
46                 msg.show();
47             }
48         });
49
50         tv3.setOnClickListener(new View.OnClickListener() {
51             @Override
52             public void onClick(View v) {
53                 et3.clearFocus();
54                 et1.requestFocus();
55
56                 String str = et3.getText().toString();
57                 Toast msg = Toast.makeText(getApplicationContext(), str, Toast.LENGTH_LONG);
58                 msg.show();
59             }
60         });
61     }
62 }

```

Recomendación

Se deja como actividad para el alumnado realizar esta app usando arrays o colecciones de objetos TextView y EditText para automatizar el registro los receptores de eventos y la posible generación dinámica de controles en la actividad.